Machine Learning for Finance

Supervised Learning - Classification

Prof. Mike Aguilar
Economics Department and Fuqua School of Business, Duke University

https:/www.linkedin.com/in/mike-aguilar-econ

Outline

1. Introduction

2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
   - Goodness of Fit

3. Multi Class Classification
   - k-Nearest Neighbors (k-NN)
   - CART - Classification and Regression Tree

Motivation

- These notes accompany the Duke Economics course *Machine Learning for Finance*.
- The course focuses on how machine learning methods can be used to design, evaluate, and implement systematic investment strategies.
- Our goal is to introduce a range of ML techniques with enough mathematical structure to build intuition, while avoiding unnecessary formalism.
- Throughout the notes we use a simple return–momentum signal as a running example. Although momentum is not always the ideal illustration for every technique, using a single, familiar signal provides a consistent thread. This is especially true when we transition to real data and empirical implementation.
- The methods we discuss are broadly applicable across many signals, asset classes, and investment settings.

Description of ML Methods

We'll use a toy momentum example as motivation

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum
- **Supervised**

Description of ML Methods

We'll use a toy momentum example as motivation

Unsupervised Learning

- **Clustering**
  Group similar assets by momentum characteristics
- **Dimension Reduction**
  Condense multiple momentum measures into fewer factors
- **Generative Modeling**
  Simulate returns to derive momentum structure

Supervised Learning

- **Classification**
  Predict positive/negative returns using momentum
- **Regression**
  Predict cross-sectional returns using momentum inputs

Time Series Modeling

- **Unsupervised**
  Group assets by temporal behavior, then measure momentum
- **Supervised**
  Forecast returns based on persistence

Outline

1. Introduction
2. Binary Classification
3. Multi Class Classification

Clustering Momentum for Trading Signal

Q: Can you design a momentum signal with CLUSTERING?

Clustering Momentum for Trading Signal

Q: Can you design a momentum signal with CLUSTERING?

A:

1. Choose features $X_{i,t}$ (e.g., multiple momentum measures, vol, etc.)

Clustering Momentum for Trading Signal

Q: Can you design a momentum signal with CLUSTERING?

A:

1. Choose features $X_{i,t}$ (e.g., multiple momentum measures, vol, etc.)
2. Standardize / transform features

Clustering Momentum for Trading Signal

Q: Can you design a momentum signal with CLUSTERING?

A:

1. Choose features $X_{i,t}$ (e.g., multiple momentum measures, vol, etc.)
2. Standardize / transform features
3. Unsupervised step: fit clusters $\mathcal{C}(X)$

Clustering Momentum for Trading Signal

Q: Can you design a momentum signal with CLUSTERING?

A:

1. Choose features $X_{i,t}$ (e.g., multiple momentum measures, vol, etc.)
2. Standardize / transform features
3. Unsupervised step: fit clusters $\mathcal{C}(X)$
4. Interpret clusters (which cluster looks like "strong momentum"?)

Clustering Momentum for Trading Signal

Q: Can you design a momentum signal with CLUSTERING?

A:

1. Choose features $X_{i,t}$ (e.g., multiple momentum measures, vol, etc.)
2. Standardize / transform features
3. Unsupervised step: fit clusters $\mathcal{C}(X)$
4. Interpret clusters (which cluster looks like "strong momentum"?)
5. Convert clusters to a trading decision (long / short / neutral)

Clustering Momentum for Trading Signal

Q: Can you design a momentum signal with CLUSTERING?

A:

1. Choose features $X_{i,t}$ (e.g., multiple momentum measures, vol, etc.)
2. Standardize / transform features
3. Unsupervised step: fit clusters $\mathcal{C}(X)$
4. Interpret clusters (which cluster looks like "strong momentum"?)
5. Convert clusters to a trading decision (long / short / neutral)
6. Evaluate out-of-sample performance

Clustering Momentum for Trading Signal

Q: Can you design a momentum signal with CLUSTERING?

A:

1. Choose features $X_{i,t}$ (e.g., multiple momentum measures, vol, etc.)
2. Standardize / transform features
3. Unsupervised step: fit clusters $\mathcal{C}(X)$
4. Interpret clusters (which cluster looks like "strong momentum"?)
5. Convert clusters to a trading decision (long / short / neutral)
6. Evaluate out-of-sample performance

Q: Where do the "labels" (long/short) come from?

Clustering Momentum for Trading Signal

Q: Can you design a momentum signal with CLUSTERING?

A:

1. Choose features $X_{i,t}$ (e.g., multiple momentum measures, vol, etc.)
2. Standardize / transform features
3. Unsupervised step: fit clusters $\mathcal{C}(X)$
4. Interpret clusters (which cluster looks like "strong momentum"?)
5. Convert clusters to a trading decision (long / short / neutral)
6. Evaluate out-of-sample performance

Q: Where do the "labels" (long/short) come from?
A: Labels are imposed ex-post.

Clustering Momentum for Trading Signal

Q: Can you design a momentum signal with CLUSTERING?

A:

1. Choose features $X_{i,t}$ (e.g., multiple momentum measures, vol, etc.)
2. Standardize / transform features
3. Unsupervised step: fit clusters $\mathcal{C}(X)$
4. Interpret clusters (which cluster looks like "strong momentum"?)
5. Convert clusters to a trading decision (long / short / neutral)
6. Evaluate out-of-sample performance

Q: Where do the "labels" (long/short) come from?
A: Labels are imposed ex-post.

By contrast, in Classification, we're going to handle the labels differently.

Clustering vs classification in a trading signal

Clustering:

Clustering vs classification in a trading signal

Clustering:

- The algorithm finds groups using only X.

Clustering vs classification in a trading signal

Clustering:

- The algorithm finds groups using only X.
- *Afterward* we assign clusters to actions (long/short) based on interpretation.

Clustering vs classification in a trading signal

Clustering:

- The algorithm finds groups using only X.
- *Afterward* we assign clusters to actions (long/short) based on interpretation.
- Instability / drift: because the "labels" are post-hoc, the mapping

$$\text{cluster ID} \rightarrow \text{trade action}$$

can *shift over time* (label switching).

Clustering vs classification in a trading signal

Clustering:

- The algorithm finds groups using only X.
- *Afterward* we assign clusters to actions (long/short) based on interpretation.
- Instability / drift: because the "labels" are post-hoc, the mapping

$$\text{cluster ID} \rightarrow \text{trade action}$$

  can *shift over time* (label switching).
- In a trading pipeline this drift is often unwanted because it makes the signal harder to interpret, harder to monitor, and less stable out-of-sample.

Clustering vs classification in a trading signal

Clustering:

- The algorithm finds groups using only X.
- *Afterward* we assign clusters to actions (long/short) based on interpretation.
- Instability / drift: because the "labels" are post-hoc, the mapping

$$\text{cluster ID} \rightarrow \text{trade action}$$

  can *shift over time* (label switching).
- In a trading pipeline this drift is often unwanted because it makes the signal harder to interpret, harder to monitor, and less stable out-of-sample.

Classification:

Clustering vs classification in a trading signal

Clustering:

- The algorithm finds groups using only X.
- *Afterward* we assign clusters to actions (long/short) based on interpretation.
- Instability / drift: because the "labels" are post-hoc, the mapping

$$\text{cluster ID} \rightarrow \text{trade action}$$

  can *shift over time* (label switching).
- In a trading pipeline this drift is often unwanted because it makes the signal harder to interpret, harder to monitor, and less stable out-of-sample.

Classification:

- We define an outcome Y (*success* / regime / event) *a priori*.

Clustering vs classification in a trading signal

Clustering:

- The algorithm finds groups using only X.
- *Afterward* we assign clusters to actions (long/short) based on interpretation.
- Instability / drift: because the "labels" are post-hoc, the mapping

$$\text{cluster ID} \rightarrow \text{trade action}$$

  can *shift over time* (label switching).
- In a trading pipeline this drift is often unwanted because it makes the signal harder to interpret, harder to monitor, and less stable out-of-sample.

Classification:

- We define an outcome Y (*success* / regime / event) *a priori*.
- We model $P(Y|X)$ or a decision rule $g(X)$ using labeled data (X, Y).

Clustering vs classification in a trading signal

Clustering:

- The algorithm finds groups using only X.
- *Afterward* we assign clusters to actions (long/short) based on interpretation.
- Instability / drift: because the "labels" are post-hoc, the mapping

$$\text{cluster ID} \rightarrow \text{trade action}$$

  can *shift over time* (label switching).
- In a trading pipeline this drift is often unwanted because it makes the signal harder to interpret, harder to monitor, and less stable out-of-sample.

Classification:

- We define an outcome Y (*success* / regime / event) *a priori*.
- We model $P(Y|X)$ or a decision rule $g(X)$ using labeled data (X, Y).
- We then predict Y (or $P(Y = 1|X)$) for new observations.

Clustering vs classification in a trading signal

Clustering:

- The algorithm finds groups using only X.
- *Afterward* we assign clusters to actions (long/short) based on interpretation.
- Instability / drift: because the "labels" are post-hoc, the mapping

$$\text{cluster ID} \rightarrow \text{trade action}$$

  can *shift over time* (label switching).
- In a trading pipeline this drift is often unwanted because it makes the signal harder to interpret, harder to monitor, and less stable out-of-sample.

Classification:

- We define an outcome Y (*success* / regime / event) *a priori*.
- We model $P(Y|X)$ or a decision rule $g(X)$ using labeled data (X, Y).
- We then predict Y (or $P(Y = 1|X)$) for new observations.

Key difference: classification trains on *fixed labeled outcomes*; clustering does not.

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?
  - A: Regimes are often defined using future information (e.g., peak-to-trough drawdowns), so they are only known ex-post.

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?
  - A: Regimes are often defined using future information (e.g., peak-to-trough drawdowns), so they are only known ex-post.
- Q: How would **clustering** handle this problem?

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?
  - A: Regimes are often defined using future information (e.g., peak-to-trough drawdowns), so they are only known ex-post.
- Q: How would **clustering** handle this problem?
- A: Use features X_t such as macro indicators, credit spreads, etc...to fit clusters, and then

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?
  - A: Regimes are often defined using future information (e.g., peak-to-trough drawdowns), so they are only known ex-post.
- Q: How would **clustering** handle this problem?
- A: Use features X_t such as macro indicators, credit spreads, etc...to fit clusters, and then interpret which clusters resemble Bull/Bear; i.e. it discovers regimes

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?
  - A: Regimes are often defined using future information (e.g., peak-to-trough drawdowns), so they are only known ex-post.
- Q: How would **clustering** handle this problem?
- A: Use features X_t such as macro indicators, credit spreads, etc...to fit clusters, and then interpret which clusters resemble Bull/Bear; i.e. it discovers regimes
- Q: How would **classification** handle this problem?

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?
  - A: Regimes are often defined using future information (e.g., peak-to-trough drawdowns), so they are only known ex-post.
- Q: How would **clustering** handle this problem?
- A: Use features X_t such as macro indicators, credit spreads, etc...to fit clusters, and then interpret which clusters resemble Bull/Bear; i.e. it discovers regimes
- Q: How would **classification** handle this problem?
- A: Define Bull/Bear apriori, then

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?
  - A: Regimes are often defined using future information (e.g., peak-to-trough drawdowns), so they are only known ex-post.
- Q: How would **clustering** handle this problem?
- A: Use features X_t such as macro indicators, credit spreads, etc...to fit clusters, and then interpret which clusters resemble Bull/Bear; i.e. it discovers regimes
- Q: How would **classification** handle this problem?
- A: Define Bull/Bear apriori, then learn the $P(Y = 1|X)$

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?
  - A: Regimes are often defined using future information (e.g., peak-to-trough drawdowns), so they are only known ex-post.
- Q: How would **clustering** handle this problem?
- A: Use features X_t such as macro indicators, credit spreads, etc...to fit clusters, and then interpret which clusters resemble Bull/Bear; i.e. it discovers regimes
- Q: How would **classification** handle this problem?
- A: Define Bull/Bear apriori, then learn the $P(Y = 1|X)$
- Q: How frame as a prediction problem? How frame as a forecasting problem?

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?
  - A: Regimes are often defined using future information (e.g., peak-to-trough drawdowns), so they are only known ex-post.
- Q: How would **clustering** handle this problem?
- A: Use features X_t such as macro indicators, credit spreads, etc...to fit clusters, and then interpret which clusters resemble Bull/Bear; i.e. it discovers regimes
- Q: How would **classification** handle this problem?
- A: Define Bull/Bear apriori, then learn the $P(Y = 1|X)$
- Q: How frame as a prediction problem? How frame as a forecasting problem?
- A: Prediction $\rightarrow P(Y_t|X_t)$

Bulls and Bears

Most natural example of classification in our trading context: regime modeling

- Suppose we knew that certain assets tend to do well in Bull markets, whereas others prosper in Bear markets.
- If we knew the regime, we would know how to invest.
  - Q: Why is even knowing the current regime difficult?
  - A: Regimes are often defined using future information (e.g., peak-to-trough drawdowns), so they are only known ex-post.
- Q: How would **clustering** handle this problem?
- A: Use features X_t such as macro indicators, credit spreads, etc...to fit clusters, and then interpret which clusters resemble Bull/Bear; i.e. it discovers regimes
- Q: How would **classification** handle this problem?
- A: Define Bull/Bear apriori, then learn the $P(Y = 1|X)$
- Q: How frame as a prediction problem? How frame as a forecasting problem?
- A: Prediction $\rightarrow P(Y_t|X_t)$ Forecast $\rightarrow P(Y_{t=1+1}|X)$

Cautionary Note About Regime Dependence

Regimes are inherently time-series objects (persistence, serial dependence).

Cautionary Note About Regime Dependence

Regimes are inherently time-series objects (persistence, serial dependence).

Today: we treat observations (X_t, Y_t) as if they are i.i.d. for the purpose of learning the *mapping* from X to Y.

- We focus on *classification methods* and their objectives (loss/risk).
- We will *not* model state persistence or transition dynamics here.

Cautionary Note About Regime Dependence

Regimes are inherently time-series objects (persistence, serial dependence).

Today: we treat observations (X_t, Y_t) as if they are i.i.d. for the purpose of learning the *mapping* from X to Y.

- We focus on *classification methods* and their objectives (loss/risk).
- We will *not* model state persistence or transition dynamics here.

Later: we revisit dependence (blocked / rolling CV, sequential decision costs, regime persistence, etc.).

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix
Q: What did we assume about β?

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix
Q: What did we assume about β?
A: Among other things, the model was linear in the parameters; ie

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix
Q: What did we assume about β?
A: Among other things, the model was linear in the parameters; ie can't permit β^2

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix
Q: What did we assume about β?
A: Among other things, the model was linear in the parameters; ie can't permit β^2
Q: What did we assume about Y_i?

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix
Q: What did we assume about β?
A: Among other things, the model was linear in the parameters; ie can't permit β^2
Q: What did we assume about Y_i?
A: Among other things Y_i was continuous.

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix
Q: What did we assume about β?
A: Among other things, the model was linear in the parameters; ie can't permit β^2
Q: What did we assume about Y_i?
A: Among other things Y_i was continuous.

If Y is **limited** (binary, multinomial, corner solution), then linear regression has structural problems:

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix
Q: What did we assume about β?
A: Among other things, the model was linear in the parameters; ie can't permit β^2
Q: What did we assume about Y_i?
A: Among other things Y_i was continuous.

If Y is **limited** (binary, multinomial, corner solution), then linear regression has structural problems:

- fitted values can violate the support of Y (e.g.,

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix
Q: What did we assume about β?
A: Among other things, the model was linear in the parameters; ie can't permit β^2
Q: What did we assume about Y_i?
A: Among other things Y_i was continuous.

If Y is **limited** (binary, multinomial, corner solution), then linear regression has structural problems:

- fitted values can violate the support of Y (e.g., probabilities < 0 or > 1),

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix
Q: What did we assume about β?
A: Among other things, the model was linear in the parameters; ie can't permit β^2
Q: What did we assume about Y_i?
A: Among other things Y_i was continuous.

If Y is **limited** (binary, multinomial, corner solution), then linear regression has structural problems:

- fitted values can violate the support of Y (e.g., probabilities < 0 or > 1),
- errors are heteroskedastic by construction,

Shortcomings of OLS

Recall linear regression:

$$Y = \beta_0 + \beta^\top X + u, \qquad \hat{\beta} = \arg\min_{\beta} \sum_{i=1}^{N} (Y_i - \beta_0 - \beta^\top X_i)^2$$

Q: What did we assume about X_i?
A: Among other things, X_i could be continuous or binary, linear or nonlinear, vector or matrix
Q: What did we assume about β?
A: Among other things, the model was linear in the parameters; ie can't permit β^2
Q: What did we assume about Y_i?
A: Among other things Y_i was continuous.

If Y is **limited** (binary, multinomial, corner solution), then linear regression has structural problems:

- fitted values can violate the support of Y (e.g., probabilities < 0 or > 1),
- errors are heteroskedastic by construction,
- decision thresholds / costs become central.

How can Y be limited?

Common limited dependent variables in finance:

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:**

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull;

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default;

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default; crisis / no crisis.

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default; crisis / no crisis.
- **Multinomial:**

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default; crisis / no crisis.
- **Multinomial:** bull / neutral / bear;

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default; crisis / no crisis.
- **Multinomial:** bull / neutral / bear; rating categories.

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default; crisis / no crisis.
- **Multinomial:** bull / neutral / bear; rating categories.
- **Ordered:**

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default; crisis / no crisis.
- **Multinomial:** bull / neutral / bear; rating categories.
- **Ordered:** ratings with order (AAA, AA, . . .).

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default; crisis / no crisis.
- **Multinomial:** bull / neutral / bear; rating categories.
- **Ordered:** ratings with order (AAA, AA, . . .).
- **Corner solutions:**

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default; crisis / no crisis.
- **Multinomial:** bull / neutral / bear; rating categories.
- **Ordered:** ratings with order (AAA, AA, . . .).
- **Corner solutions:** portfolio weight at 0 for many assets; participation decisions.

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default; crisis / no crisis.
- **Multinomial:** bull / neutral / bear; rating categories.
- **Ordered:** ratings with order (AAA, AA, . . .).
- **Corner solutions:** portfolio weight at 0 for many assets; participation decisions.

How can Y be limited?

Common limited dependent variables in finance:

- **Binary:** bear/bull; default / no default; crisis / no crisis.
- **Multinomial:** bull / neutral / bear; rating categories.
- **Ordered:** ratings with order (AAA, AA, . . .).
- **Corner solutions:** portfolio weight at 0 for many assets; participation decisions.

We will begin with binary, then move to multinomial, but likely won't cover Ordered or Corner Solutions.

Outline

1. Introduction

2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
   - Goodness of Fit

3. Multi Class Classification

Outline

1. Introduction

2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
   - Goodness of Fit

3. Multi Class Classification

What is the Probability of a Bear Market?

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{otherwise} \end{cases} \qquad X_t \in \mathbb{R}^K \text{ (features)}$$

What is the Probability of a Bear Market?

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{otherwise} \end{cases} \qquad X_t \in \mathbb{R}^K \text{ (features)}$$

As mentioned earlier, we can treat this as prediction or forecasting.

What is the Probability of a Bear Market?

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{otherwise} \end{cases} \qquad X_t \in \mathbb{R}^K \text{ (features)}$$

As mentioned earlier, we can treat this as prediction or forecasting.
Linear Probability Model:

$$P(Y_t = 1|X_t) = \mathbb{E}[Y_t|X_t] = \beta_0 + \beta^\top X_t$$

What is the Probability of a Bear Market?

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{otherwise} \end{cases} \qquad X_t \in \mathbb{R}^K \text{ (features)}$$

As mentioned earlier, we can treat this as prediction or forecasting.
Linear Probability Model:

$$P(Y_t = 1|X_t) = \mathbb{E}[Y_t|X_t] = \beta_0 + \beta^\top X_t$$

Estimate by OLS:

$$(\hat{\beta}_0, \hat{\beta}) = \arg\min_{\beta_0, \beta} \sum_{t=1}^{T} (Y_t - \beta_0 - \beta^\top X_t)^2$$

What does the LPM output mean?

Output: fitted values $\hat{p}_t = \hat{\beta}_0 + \hat{\beta}^\top X_t$.

What does the LPM output mean?

Output: fitted values $\hat{p}_t = \hat{\beta}_0 + \hat{\beta}^\top X_t$.

Interpretation:

- $\hat{p}_t$ is a *probability forecast* (not a regime label).

What does the LPM output mean?

Output: fitted values $\hat{p}_t = \hat{\beta}_0 + \hat{\beta}^\top X_t$.

Interpretation:

- $\hat{p}_t$ is a *probability forecast* (not a regime label).
- To produce a regime prediction/forecast, choose

What does the LPM output mean?

Output: fitted values $\hat{p}_t = \hat{\beta}_0 + \hat{\beta}^\top X_t$.

Interpretation:

- $\hat{p}_t$ is a *probability forecast* (not a regime label).
- To produce a regime prediction/forecast, choose a threshold τ:

$$\hat{Y}_t(\tau) = \mathbf{1}\{\hat{p}_t \geq \tau\}$$

What does the LPM output mean?

Output: fitted values $\hat{p}_t = \hat{\beta}_0 + \hat{\beta}^\top X_t$.

Interpretation:

- $\hat{p}_t$ is a *probability forecast* (not a regime label).
- To produce a regime prediction/forecast, choose a threshold τ:

$$\hat{Y}_t(\tau) = \mathbf{1}\{\hat{p}_t \geq \tau\}$$

- e.g. If probability of Bear $> 60\%$ we classify as Bear market.

Weaknesses of the LPM

1. **Support violations:**

Weaknesses of the LPM

1. **Support violations:** $\hat{p}_t$ can be < 0 or > 1.

Weaknesses of the LPM

1. **Support violations:** $\hat{p}_t$ can be < 0 or > 1.
2. **Heteroskedasticity is structural:**

Weaknesses of the LPM

1. **Support violations:** $\hat{p}_t$ can be < 0 or > 1.
2. **Heteroskedasticity is structural:**

$$\mathrm{Var}(Y_t|X_t) = p_t(1 - p_t)$$

So OLS standard errors need robust corrections.

Weaknesses of the LPM

1. **Support violations:** $\hat{p}_t$ can be < 0 or > 1.
2. **Heteroskedasticity is structural:**

$$\mathrm{Var}(Y_t|X_t) = p_t(1 - p_t)$$

   So OLS standard errors need robust corrections.
3. **Rare events:** if bear is rare, accuracy can be misleading

Weaknesses of the LPM

1. **Support violations:** $\hat{p}_t$ can be < 0 or > 1.
2. **Heteroskedasticity is structural:**

$$\mathrm{Var}(Y_t|X_t) = p_t(1 - p_t)$$

   So OLS standard errors need robust corrections.
3. **Rare events:** if bear is rare, accuracy can be misleading
4. **Time dependence (caveat):** regimes are persistent; naive i.i.d. inference is not appropriate for time-series standard errors.

Weaknesses of the LPM

1. **Support violations:** $\hat{p}_t$ can be < 0 or > 1.
2. **Heteroskedasticity is structural:**

$$\mathrm{Var}(Y_t|X_t) = p_t(1 - p_t)$$

   So OLS standard errors need robust corrections.
3. **Rare events:** if bear is rare, accuracy can be misleading
4. **Time dependence (caveat):** regimes are persistent; naive i.i.d. inference is not appropriate for time-series standard errors.

Conclusion: LPM is a useful baseline, but we want a model with $0 < p < 1$ by construction.

Outline

1. Introduction

2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
   - Goodness of Fit

3. Multi Class Classification

What is the Probability of a Bear Market? (Logit/Probit)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

What is the Probability of a Bear Market? (Logit/Probit)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

The Linear Probability Model assumes

$$P(Y_t = 1 \mid x_t) = \beta_0 + \beta^\top x_t$$

What is the Probability of a Bear Market? (Logit/Probit)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

The Linear Probability Model assumes

$$P(Y_t = 1 \mid x_t) = \beta_0 + \beta^\top x_t$$

Problem

The linear score

$$\beta_0 + \beta^\top x_t \in (-\infty, \infty)$$

so predicted probabilities can be invalid (e.g. $p_t < 0$ or $p_t > 1$).

What is the Probability of a Bear Market? (Logit/Probit)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

The Linear Probability Model assumes

$$P(Y_t = 1 \mid x_t) = \beta_0 + \beta^\top x_t$$

Problem

The linear score

$$\beta_0 + \beta^\top x_t \in (-\infty, \infty)$$

so predicted probabilities can be invalid (e.g. $p_t < 0$ or $p_t > 1$).

Idea

Transform the linear score into a valid probability.

Logit and Probit

Define the linear score

$$\eta_t = \beta_0 + \beta^\top x_t$$

Logit and Probit

Define the linear score

$$\eta_t = \beta_0 + \beta^\top x_t$$

Map the score into a probability via link function G:

$$P(Y_t = 1 \mid x_t) = G(\eta_t)$$

where

$$G : \mathbb{R} \to (0, 1)$$

Logit and Probit

Define the linear score

$$\eta_t = \beta_0 + \beta^\top x_t$$

Map the score into a probability via link function G:

$$P(Y_t = 1 \mid x_t) = G(\eta_t)$$

where

$$G : \mathbb{R} \to (0, 1)$$

Two common choices for the link function

- Logit

$$G(\eta) = \frac{1}{1 + e^{-\eta}}$$

Logit and Probit

Define the linear score

$$\eta_t = \beta_0 + \beta^\top x_t$$

Map the score into a probability via link function G:

$$P(Y_t = 1 \mid x_t) = G(\eta_t)$$

where

$$G : \mathbb{R} \to (0, 1)$$

Two common choices for the link function

- Logit

$$G(\eta) = \frac{1}{1 + e^{-\eta}}$$

- Probit

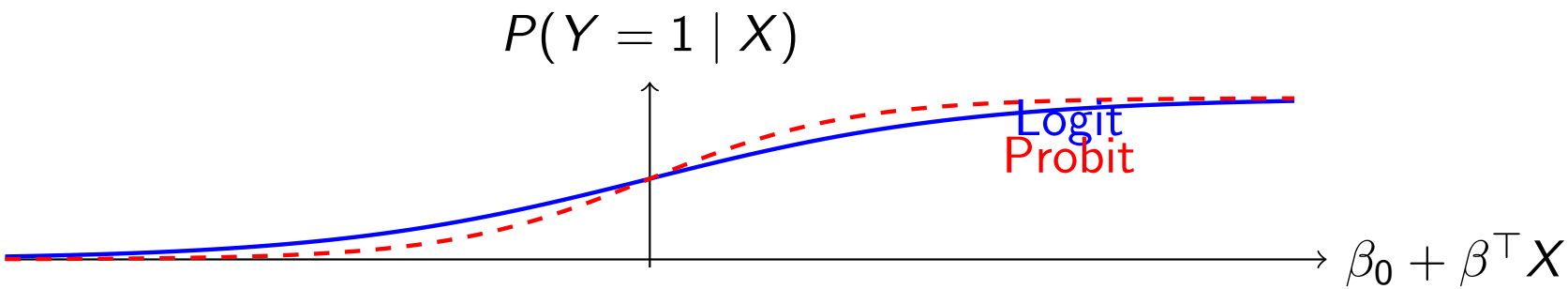
$$G(\eta) = \Phi(\eta)$$

Logit vs Probit

Both models map a linear score into a probability $P(Y = 1 \mid X) = G(\beta_0 + \beta^\top X)$

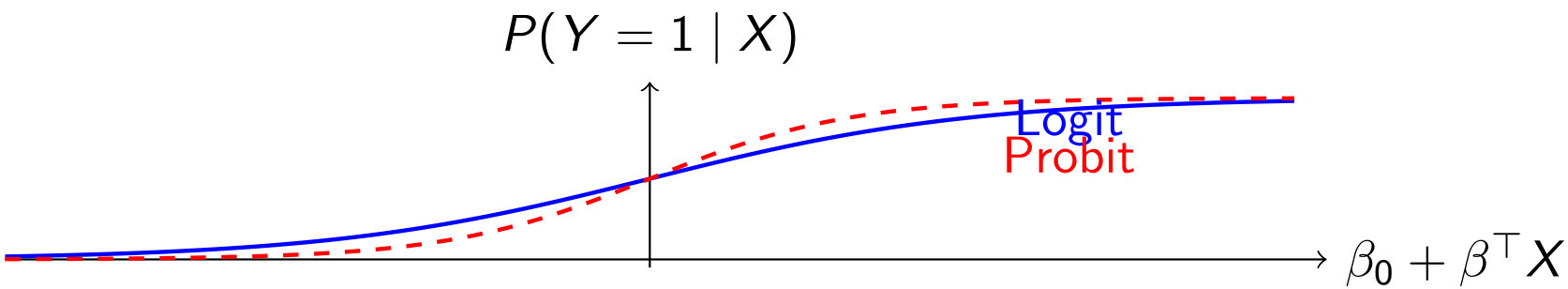
Logit vs Probit

Both models map a linear score into a probability $P(Y = 1 \mid X) = G(\beta_0 + \beta^\top X)$



Logit vs Probit

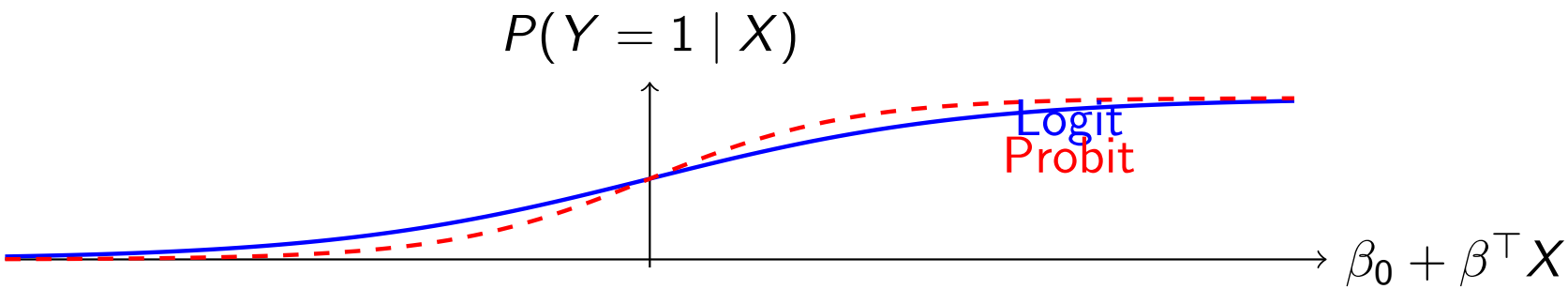
Both models map a linear score into a probability $P(Y = 1 \mid X) = G(\beta_0 + \beta^\top X)$



- Both models transform a linear score into a probability.

Logit vs Probit

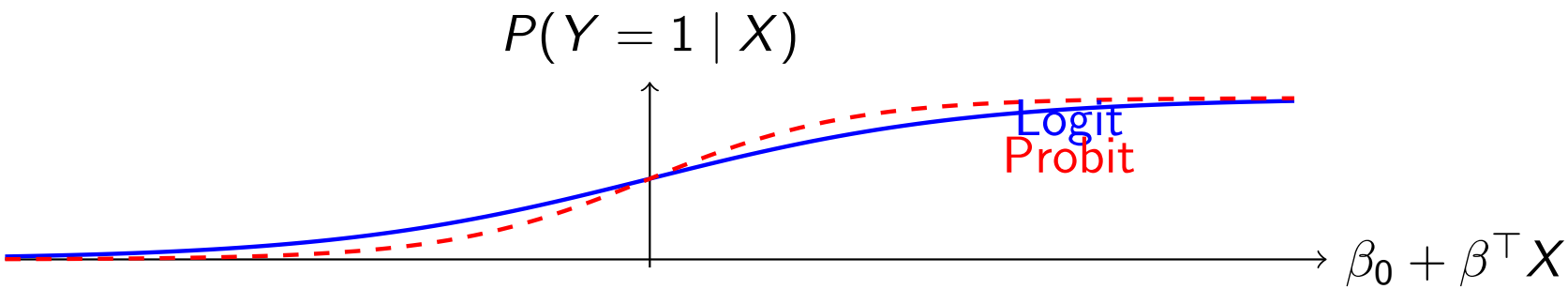
Both models map a linear score into a probability $P(Y = 1 \mid X) = G(\beta_0 + \beta^\top X)$



- Both models transform a linear score into a probability.
- The two curves are nearly identical.

Logit vs Probit

Both models map a linear score into a probability $P(Y = 1 \mid X) = G(\beta_0 + \beta^\top X)$



- Both models transform a linear score into a probability.
- The two curves are nearly identical.
- In practice, logit and probit typically produce very similar predicted probabilities.

Estimation by Maximum Likelihood

Let $p_t(\beta) = G(\beta_0 + \beta^\top x_t)$.

Estimation by Maximum Likelihood

Let $p_t(\beta) = G(\beta_0 + \beta^\top x_t)$.
Assume conditional Bernoulli:

$$\mathbb{P}(Y_t \mid x_t; \beta) = p_t(\beta)^{Y_t} \left[1 - p_t(\beta)\right]^{1-Y_t}$$

Estimation by Maximum Likelihood

Let $p_t(\beta) = G(\beta_0 + \beta^\top x_t)$.
Assume conditional Bernoulli:

$$\mathbb{P}(Y_t \mid x_t; \beta) = p_t(\beta)^{Y_t} \left[1 - p_t(\beta)\right]^{1-Y_t}$$

Log-likelihood:

$$\ell(\beta) = \sum_{t=1}^{T} \Big(Y_t \log p_t(\beta) + (1 - Y_t) \log(1 - p_t(\beta)) \Big)$$

Estimation by Maximum Likelihood

Let $p_t(\beta) = G(\beta_0 + \beta^\top x_t)$.
Assume conditional Bernoulli:

$$\mathbb{P}(Y_t \mid x_t; \beta) = p_t(\beta)^{Y_t} \left[1 - p_t(\beta)\right]^{1 - Y_t}$$

Log-likelihood:

$$\ell(\beta) = \sum_{t=1}^{T} \Big(Y_t \log p_t(\beta) + (1 - Y_t) \log(1 - p_t(\beta)) \Big)$$

$$\hat{\beta} = \arg\max_{\beta} \ell(\beta)$$

Estimation by Maximum Likelihood

Let $p_t(\beta) = G(\beta_0 + \beta^\top x_t)$.
Assume conditional Bernoulli:

$$\mathbb{P}(Y_t \mid x_t; \beta) = p_t(\beta)^{Y_t} \left[1 - p_t(\beta)\right]^{1-Y_t}$$

Log-likelihood:

$$\ell(\beta) = \sum_{t=1}^{T} \Big(Y_t \log p_t(\beta) + (1 - Y_t) \log(1 - p_t(\beta)) \Big)$$

$$\hat{\beta} = \arg\max_{\beta} \ell(\beta)$$

Estimated numerically.

Predictions

Logit and probit models estimate the probability of Bear:

$$p_t = P(Y_t = 1 \mid x_t)$$

Predictions

Logit and probit models estimate the probability of Bear:

$$p_t = P(Y_t = 1 \mid x_t)$$

Logit prediction

$$p_t = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x_t)}}$$

Predictions

Logit and probit models estimate the probability of Bear:

$$p_t = P(Y_t = 1 \mid x_t)$$

Logit prediction

$$p_t = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x_t)}}$$

Probit prediction

$$p_t = \Phi(\beta_0 + \beta^\top x_t)$$

Predictions

Logit and probit models estimate the probability of Bear:

$$p_t = P(Y_t = 1 \mid x_t)$$

Logit prediction

$$p_t = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x_t)}}$$

Probit prediction

$$p_t = \Phi(\beta_0 + \beta^\top x_t)$$

Example (Bear market case study)

- Suppose $\beta_0 + \beta^\top x_t = -1.39$

Predictions

Logit and probit models estimate the probability of Bear:

$$p_t = P(Y_t = 1 \mid x_t)$$

Logit prediction

$$p_t = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x_t)}}$$

Probit prediction

$$p_t = \Phi(\beta_0 + \beta^\top x_t)$$

Example (Bear market case study)

- Suppose $\beta_0 + \beta^\top x_t = -1.39$
- Logit: $p_t \approx 0.20$; i.e. 20% probability of a Bear

Predictions

Logit and probit models estimate the probability of Bear:

$$p_t = P(Y_t = 1 \mid x_t)$$

Logit prediction

$$p_t = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x_t)}}$$

Probit prediction

$$p_t = \Phi(\beta_0 + \beta^\top x_t)$$

Example (Bear market case study)

- Suppose $\beta_0 + \beta^\top x_t = -1.39$
- Logit: $p_t \approx 0.20$; i.e. 20% probability of a Bear
- Probit: $p_t \approx 0.18$; i.e. 18% probability of a Bear

Predictions

Logit and probit models estimate the probability of Bear:

$$p_t = P(Y_t = 1 \mid x_t)$$

Logit prediction

$$p_t = \frac{1}{1 + e^{-(\beta_0 + \beta^\top x_t)}}$$

Probit prediction

$$p_t = \Phi(\beta_0 + \beta^\top x_t)$$

Example (Bear market case study)

- Suppose $\beta_0 + \beta^\top x_t = -1.39$
- Logit: $p_t \approx 0.20$; i.e. 20% probability of a Bear
- Probit: $p_t \approx 0.18$; i.e. 18% probability of a Bear

The model outputs a probability of Bear conditional on the observed features x_t.

Converting Probability to Model Scale

Suppose that using the observed features x_t the model predicts a baseline probability of Bear $p_t = 0.20$

Converting Probability to Model Scale

Suppose that using the observed features x_t the model predicts a baseline probability of Bear $p_t = 0.20$

Logit

Convert probability to odds:

$$\text{odds} = \frac{p}{1-p} = \frac{0.20}{0.80} = 0.25$$

Converting Probability to Model Scale

Suppose that using the observed features x_t the model predicts a baseline probability of Bear $p_t = 0.20$

Logit

Convert probability to odds:

$$\text{odds} = \frac{p}{1-p} = \frac{0.20}{0.80} = 0.25$$

Probit

Convert probability to the latent score:

$$\eta_t = \Phi^{-1}(0.20) \approx -0.84$$

Converting Probability to Model Scale

Suppose that using the observed features x_t the model predicts a baseline probability of Bear $p_t = 0.20$

Logit

Convert probability to odds:

$$\text{odds} = \frac{p}{1-p} = \frac{0.20}{0.80} = 0.25$$

Probit

Convert probability to the latent score:

$$\eta_t = \Phi^{-1}(0.20) \approx -0.84$$

These are simply different ways of representing the same probability.

Feature Importance: Credit Spreads Example

Suppose the estimated coefficient on credit spreads is

$$\hat{\beta}_{\Delta CS} = 0.8$$

Feature Importance: Credit Spreads Example

Suppose the estimated coefficient on credit spreads is

$$\hat{\beta}_{\Delta CS} = 0.8$$

	Logit	**Probit**
Baseline probability	0.20	0.20

Feature Importance: Credit Spreads Example

Suppose the estimated coefficient on credit spreads is

$$\hat{\beta}_{\Delta CS} = 0.8$$

	Logit	**Probit**
Baseline probability	0.20	0.20
Model scale	Odds = 0.25	Score = -0.84

Feature Importance: Credit Spreads Example

Suppose the estimated coefficient on credit spreads is

$$\hat{\beta}_{\Delta CS} = 0.8$$

	Logit	**Probit**
Baseline probability	0.20	0.20
Model scale	Odds = 0.25	Score = -0.84
After $\Delta CS = +1$	Odds $\times e^{0.8}$	Score +0.8

Feature Importance: Credit Spreads Example

Suppose the estimated coefficient on credit spreads is

$$\hat{\beta}_{\Delta CS} = 0.8$$

	Logit	**Probit**
Baseline probability	0.20	0.20
Model scale	Odds = 0.25	Score = -0.84
After $\Delta CS = +1$	Odds $\times e^{0.8}$	Score +0.8
New value	0.56	-0.04

Feature Importance: Credit Spreads Example

Suppose the estimated coefficient on credit spreads is

$$\hat{\beta}_{\Delta CS} = 0.8$$

	Logit	**Probit**
Baseline probability	0.20	0.20
Model scale	Odds = 0.25	Score = -0.84
After $\Delta CS = +1$	Odds $\times e^{0.8}$	Score +0.8
New value	0.56	-0.04
Converted to probability	0.36	0.48

Feature Importance: Credit Spreads Example

Suppose the estimated coefficient on credit spreads is

$$\hat{\beta}_{\Delta CS} = 0.8$$

	Logit	**Probit**
Baseline probability	0.20	0.20
Model scale	Odds = 0.25	Score = -0.84
After $\Delta CS = +1$	Odds $\times e^{0.8}$	Score +0.8
New value	0.56	-0.04
Converted to probability	0.36	0.48

An increase in credit spreads substantially raises the probability of a Bear regime in both models.

How Features Shift the Probability of Bear

The model first computes a linear score:

$$\eta_t = \beta_0 + \beta^\top x_t$$

How Features Shift the Probability of Bear

The model first computes a linear score:

$$\eta_t = \beta_0 + \beta^\top x_t$$

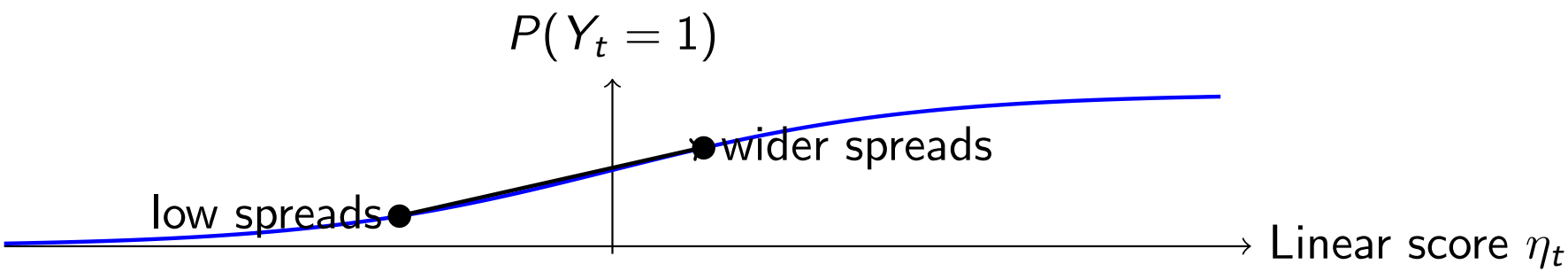
An increase in a feature (e.g., credit spreads) changes η_t, which moves the predicted probability along the S-curve.

How Features Shift the Probability of Bear

The model first computes a linear score:

$$\eta_t = \beta_0 + \beta^\top x_t$$

An increase in a feature (e.g., credit spreads) changes η_t, which moves the predicted probability along the S-curve.

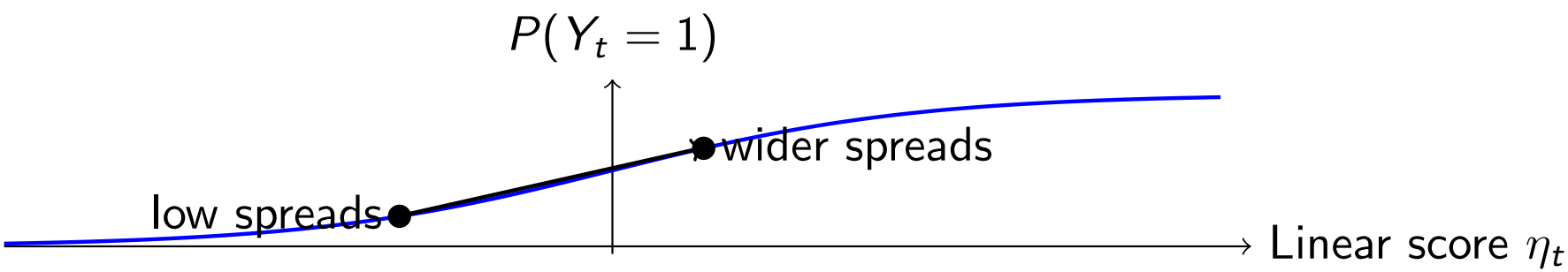


How Features Shift the Probability of Bear

The model first computes a linear score:

$$\eta_t = \beta_0 + \beta^\top x_t$$

An increase in a feature (e.g., credit spreads) changes η_t, which moves the predicted probability along the S-curve.



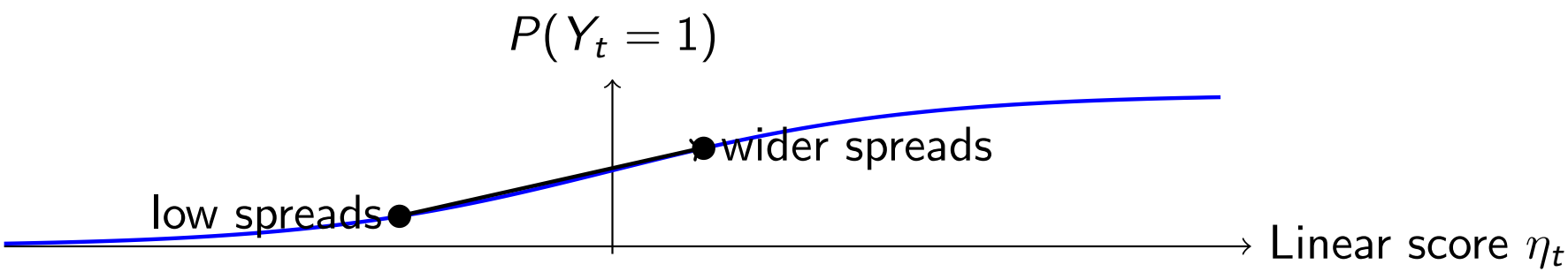
- Features affect the probability of Bear by shifting the linear score η_t.

How Features Shift the Probability of Bear

The model first computes a linear score:

$$\eta_t = \beta_0 + \beta^\top x_t$$

An increase in a feature (e.g., credit spreads) changes η_t, which moves the predicted probability along the S-curve.



- Features affect the probability of Bear by shifting the linear score η_t.
- Because the curve is nonlinear, the same change in η_t can produce different probability changes depending on where you start.

Summary: LPM vs Logit/Probit

	LPM	**Logit/Probit**
Range of $\hat{p}$	Unbounded	$(0, 1)$
Estimation	OLS	MLE
Interpretation	Linear in X	Nonlinear in X
Errors	Heteroskedastic	Bernoulli variance
Best use	Baseline	Standard probabilistic classifier

Outline

1. Introduction

2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
   - Goodness of Fit

3. Multi Class Classification

Bear Market Classification with SVM

Let

$$Y_t = \begin{cases} +1 & \text{bear market at time } t \\ -1 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Bear Market Classification with SVM

Let

$$Y_t = \begin{cases} +1 & \text{bear market at time } t \\ -1 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Goal: learn a rule that maps features to tomorrow regime:

$$\widehat{Y}_t = \operatorname{sign}\left(w^\top x_t + b\right)$$

Bear Market Classification with SVM

Let

$$Y_t = \begin{cases} +1 & \text{bear market at time } t \\ -1 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Goal: learn a rule that maps features to tomorrow regime:

$$\widehat{Y}_t = \operatorname{sign}\left(w^\top x_t + b\right)$$

- Each observation is a **time period** t, represented by a **feature vector** x_t.

Bear Market Classification with SVM

Let

$$Y_t = \begin{cases} +1 & \text{bear market at time } t \\ -1 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Goal: learn a rule that maps features to tomorrow regime:

$$\widehat{Y}_t = \operatorname{sign}\left(w^\top x_t + b\right)$$

- Each observation is a **time period** t, represented by a **feature vector** x_t.
- SVM chooses (w, b) to separate Bear vs Not-Bear in **feature space**.

Bear Market Classification with SVM

Let

$$Y_t = \begin{cases} +1 & \text{bear market at time } t \\ -1 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Goal: learn a rule that maps features to tomorrow regime:

$$\widehat{Y}_t = \operatorname{sign}\left(w^\top x_t + b\right)$$

- Each observation is a **time period** t, represented by a **feature vector** x_t.
- SVM chooses (w, b) to separate Bear vs Not-Bear in **feature space**.
- As noted earlier, we could cast this either as prediction or forecasting type problem.

Bear Market Classification with SVM

Let

$$Y_{t+1} = \begin{cases} +1 & \text{bear market at time } t+1 \\ -1 & \text{not-bear at time } t+1 \end{cases} \qquad x_t \in \mathbb{R}^K$$

Bear Market Classification with SVM

Let

$$Y_{t+1} = \begin{cases} +1 & \text{bear market at time } t+1 \\ -1 & \text{not-bear at time } t+1 \end{cases} \qquad x_t \in \mathbb{R}^K$$

Goal: learn a rule that maps today's features to tomorrow's regime:

$$\widehat{Y}_{t+1} = \operatorname{sign}\left(w^\top x_t + b\right)$$

Bear Market Classification with SVM

Let

$$Y_{t+1} = \begin{cases} +1 & \text{bear market at time } t+1 \\ -1 & \text{not-bear at time } t+1 \end{cases} \qquad x_t \in \mathbb{R}^K$$

Goal: learn a rule that maps today's features to tomorrow's regime:

$$\widehat{Y}_{t+1} = \operatorname{sign}\left(w^\top x_t + b\right)$$

- Each observation is a **time period** t, represented by a **feature vector** x_t.

Bear Market Classification with SVM

Let

$$Y_{t+1} = \begin{cases} +1 & \text{bear market at time } t+1 \\ -1 & \text{not-bear at time } t+1 \end{cases} \qquad x_t \in \mathbb{R}^K$$

Goal: learn a rule that maps today's features to tomorrow's regime:

$$\widehat{Y}_{t+1} = \operatorname{sign}\left(w^\top x_t + b\right)$$

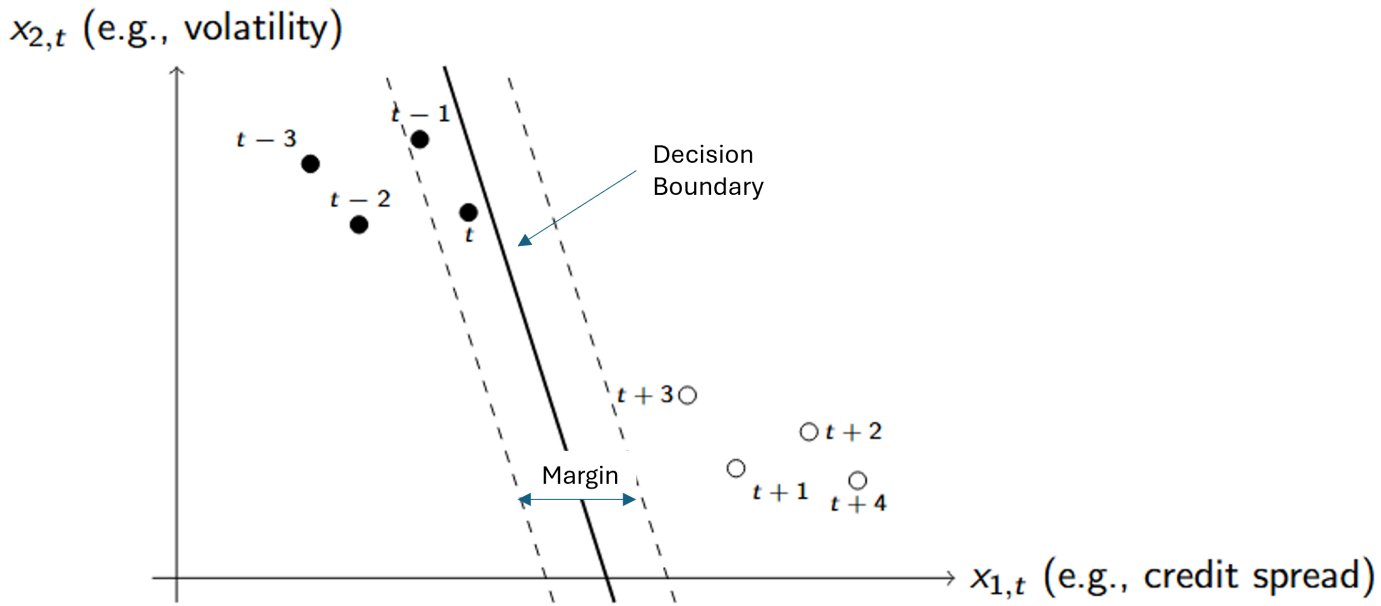
- Each observation is a **time period** t, represented by a **feature vector** x_t.
- SVM chooses (w, b) to separate Bear vs Not-Bear in **feature space**.

SVM: separating regimes in feature space

Each dot is a time period t mapped to features x_t

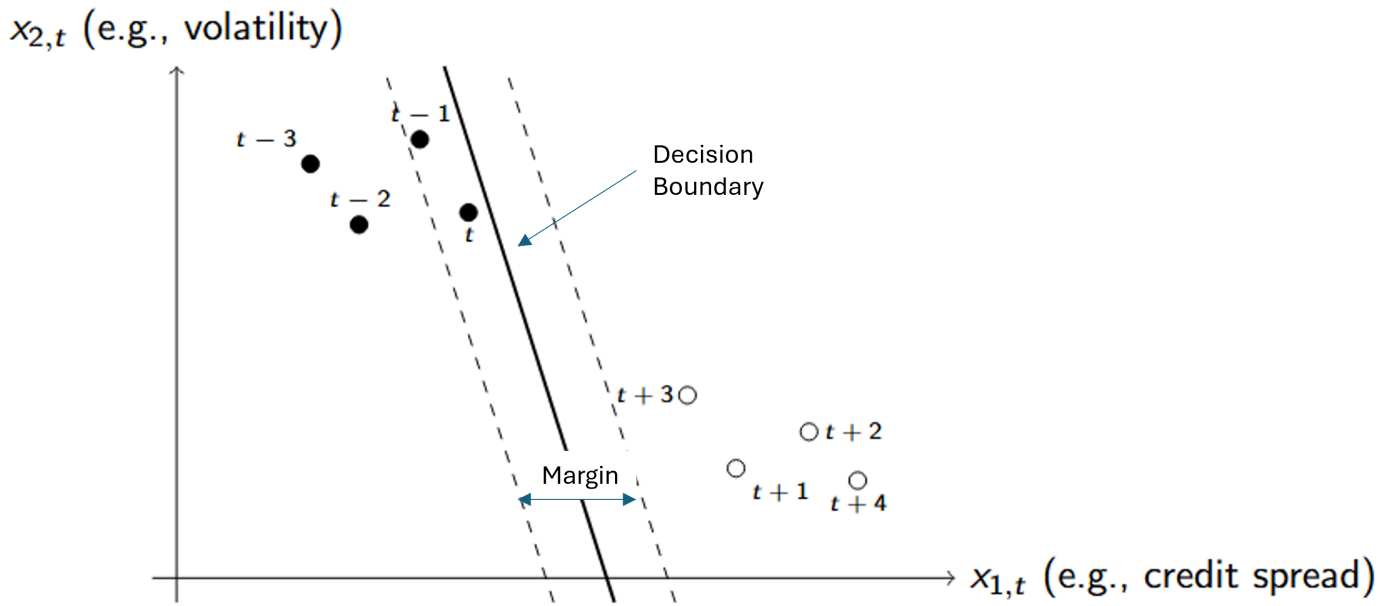
SVM: separating regimes in feature space

Each dot is a time period t mapped to features x_t



SVM: separating regimes in feature space

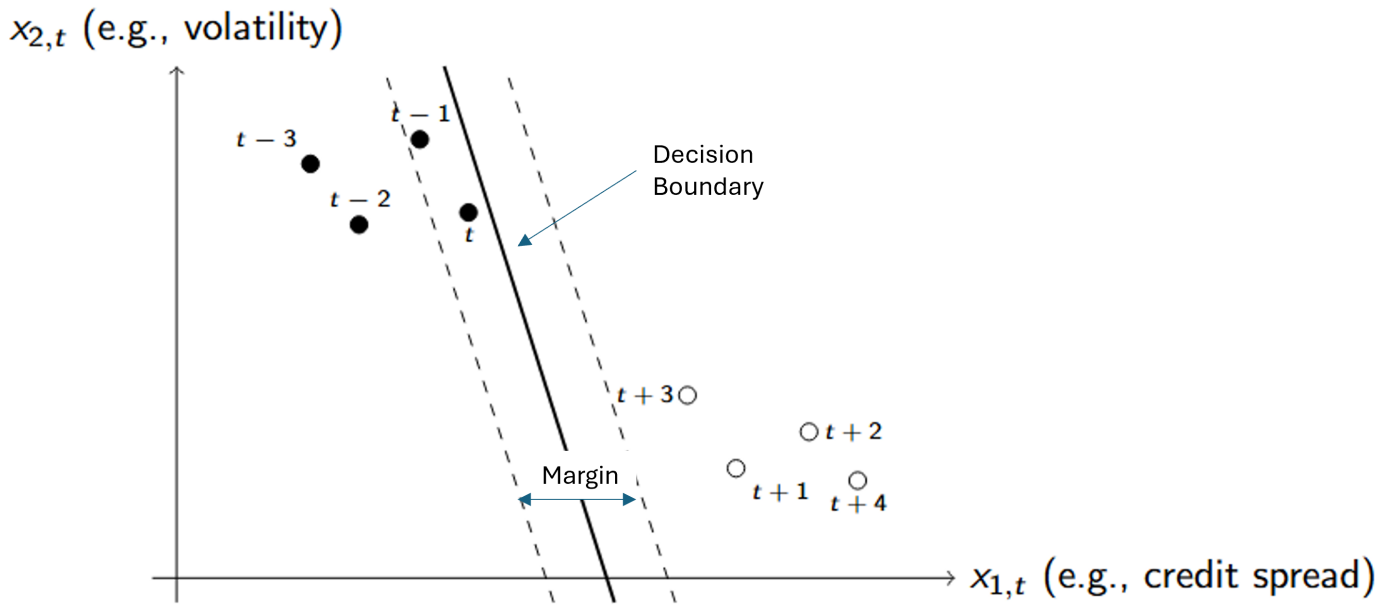
Each dot is a time period t mapped to features x_t



- SVM picks a hyperplane with the largest margin w

SVM: separating regimes in feature space

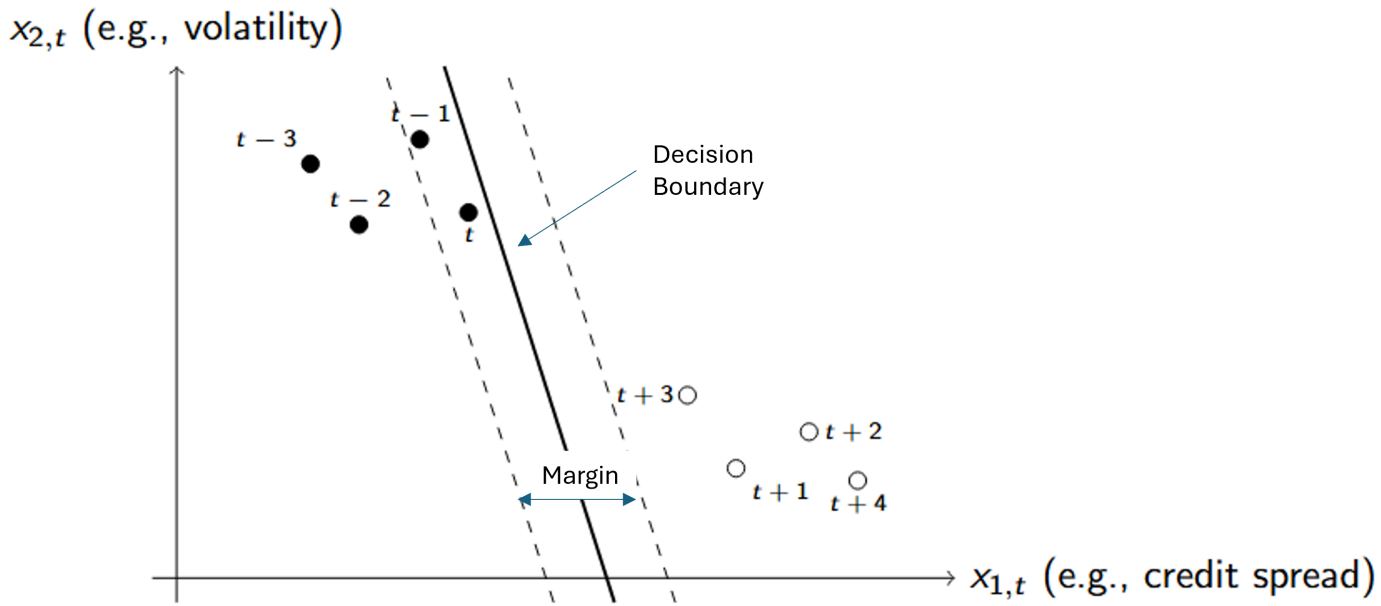
Each dot is a time period t mapped to features x_t



- SVM picks a hyperplane with the largest margin w
- Here we assume a linear decision boundary in feature space.

SVM: separating regimes in feature space

Each dot is a time period t mapped to features x_t



- SVM picks a hyperplane with the largest margin w
- Here we assume a linear decision boundary in feature space.
- If separation requires curvature, SVM can be extended using kernel methods.

Hard-margin

If data are linearly separable, we solve:

$$\min_{w,b} \ \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1, \ i = 1, \ldots, n$$

Hard-margin

If data are linearly separable, we solve:

$$\min_{w,b} \ \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1, \ i = 1, \ldots, n$$

Margin: distance between the two supporting hyperplanes is $2/\|w\|$. So minimizing $\|w\|$ maximizes the margin.

Hard-margin

If data are linearly separable, we solve:

$$\min_{w,b} \ \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1, \ i = 1, \ldots, n$$

Margin: distance between the two supporting hyperplanes is $2/\|w\|$. So minimizing $\|w\|$ maximizes the margin.

Linearly separable means there exists (w, b) with $y_i(w^\top x_i + b) > 0$ for all i.

Difficulty finding linear separability

In bull/bear regimes, perfect separability is unlikely because

- noise:

Difficulty finding linear separability

In bull/bear regimes, perfect separability is unlikely because

- noise: returns can be very noise within regimes; feature lead / lags can be variable

Difficulty finding linear separability

In bull/bear regimes, perfect separability is unlikely because

- noise: returns can be very noise within regimes; feature lead / lags can be variable
- overlap:

Difficulty finding linear separability

In bull/bear regimes, perfect separability is unlikely because

- noise: returns can be very noise within regimes; feature lead / lags can be variable
- overlap: the distributions of features for bulls often overlaps with that of bears

Difficulty finding linear separability

In bull/bear regimes, perfect separability is unlikely because

- noise: returns can be very noise within regimes; feature lead / lags can be variable
- overlap: the distributions of features for bulls often overlaps with that of bears
- label ambiguity:

Difficulty finding linear separability

In bull/bear regimes, perfect separability is unlikely because

- noise: returns can be very noise within regimes; feature lead / lags can be variable
- overlap: the distributions of features for bulls often overlaps with that of bears
- label ambiguity: regimes definitions can be arbitrary so to adjacent time periods can have nearly identical features

Soft-margin SVM

Allow violations via slack variables $\xi_i \geq 0$:

$$\min_{w,b,\xi} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i$$

Soft-margin SVM

Allow violations via slack variables $\xi_i \geq 0$:

$$\min_{w,b,\xi} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i$$

Tradeoff:

- Larger C:

Soft-margin SVM

Allow violations via slack variables $\xi_i \geq 0$:

$$\min_{w,b,\xi} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i$$

Tradeoff:

- Larger C: penalize violations more $\Rightarrow$ lower bias, higher variance (less regularization)

Soft-margin SVM

Allow violations via slack variables $\xi_i \geq 0$:

$$\min_{w,b,\xi} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i$$

Tradeoff:

- Larger C: penalize violations more $\Rightarrow$ lower bias, higher variance (less regularization)
- Smaller C:

Soft-margin SVM

Allow violations via slack variables $\xi_i \geq 0$:

$$\min_{w,b,\xi} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i$$

Tradeoff:

- Larger C: penalize violations more $\Rightarrow$ lower bias, higher variance (less regularization)
- Smaller C: allow violations $\Rightarrow$ more regularization, potentially better generalization

Soft-margin SVM

Allow violations via slack variables $\xi_i \geq 0$:

$$\min_{w,b,\xi} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i$$

Tradeoff:

- Larger C: penalize violations more $\Rightarrow$ lower bias, higher variance (less regularization)
- Smaller C: allow violations $\Rightarrow$ more regularization, potentially better generalization

Loss view: soft-margin SVM is equivalent to regularized empirical risk minimization with hinge loss:

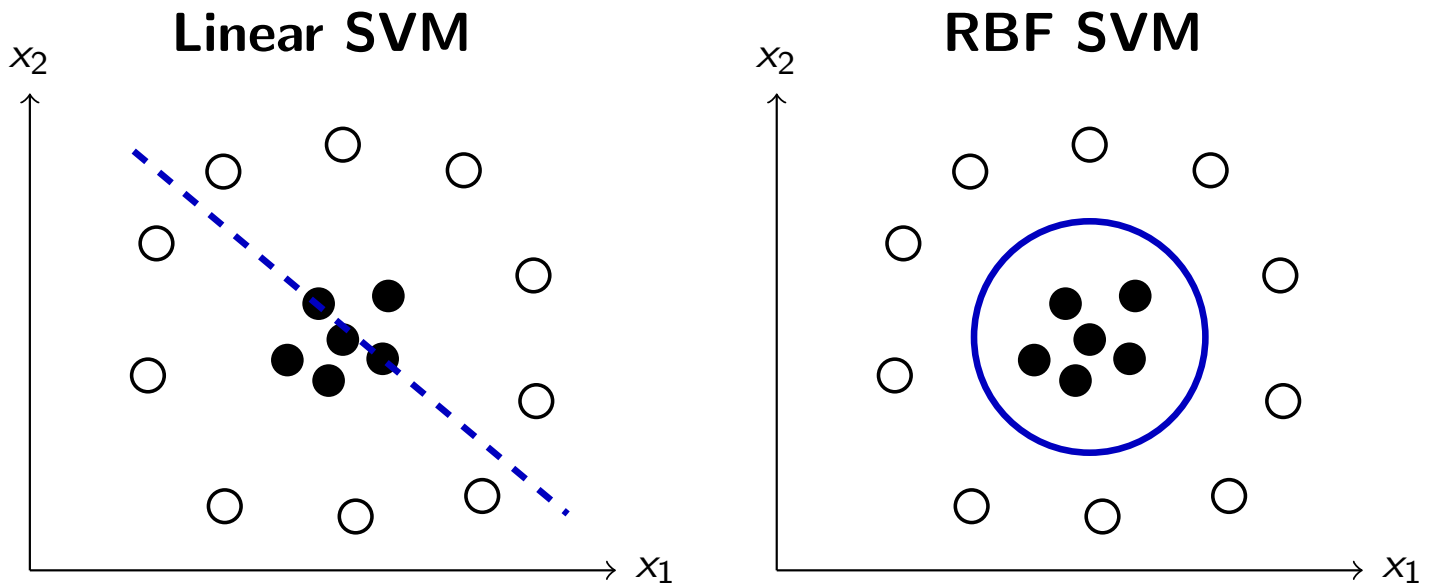
$$\min_{w,b} \ \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\max\{0, 1 - y_i(w^\top x_i + b)\}$$

Why a Linear SVM May Be Insufficient

Problem: Bear and Not-Bear observations need not be separated by a straight line in feature space.

Why a Linear SVM May Be Insufficient

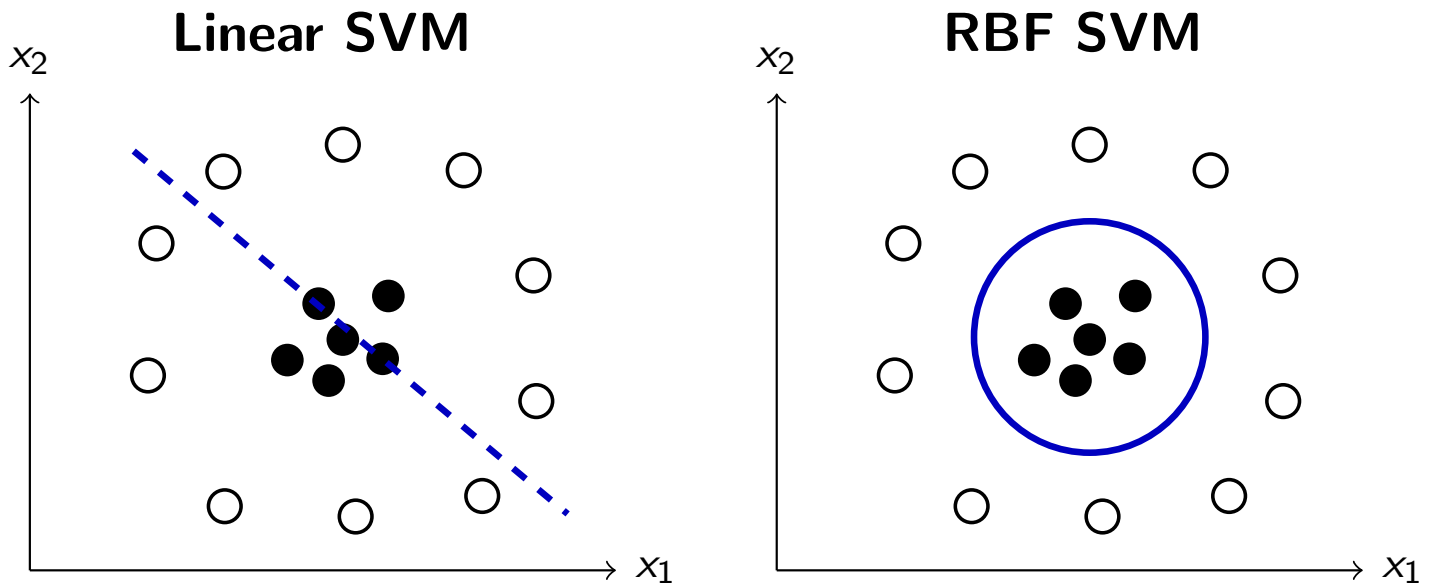
Problem: Bear and Not-Bear observations need not be separated by a straight line in feature space.



- A **linear SVM** uses $\hat{Y}_{t+1} = \text{sign}(w^\top x_t + b)$; a soft margin permits violations, *not curvature*.

Why a Linear SVM May Be Insufficient

Problem: Bear and Not-Bear observations need not be separated by a straight line in feature space.



- A **linear SVM** uses $\hat{Y}_{t+1} = \text{sign}(w^\top x_t + b)$; a soft margin permits violations, *not curvature*.
- A **kernel SVM** can learn a curved boundary, but additional flexibility may overfit.

Stylized: x_1 = credit spreads, x_2 = volatility; filled = Bear, open = Not-Bear.

Nonlinear SVM

Transform: Fit a linear SVM using nonlinear features $\phi(x)$:

$$f(x_t) = \langle w, \phi(x_t) \rangle + b.$$

Nonlinear SVM

Transform: Fit a linear SVM using nonlinear features $\phi(x)$:

$$f(x_t) = \langle w, \phi(x_t) \rangle + b.$$

Why dot products? At the optimum, SVM weights are sums of transformed features:

$$w = \sum_{s=1}^{n} \alpha_s y_s \phi(x_s), \qquad 0 \leq \alpha_s \leq C.$$

Nonlinear SVM

Transform: Fit a linear SVM using nonlinear features $\phi(x)$:

$$f(x_t) = \langle w, \phi(x_t) \rangle + b.$$

Why dot products? At the optimum, SVM weights are sums of transformed features:

$$w = \sum_{s=1}^{n} \alpha_s y_s \phi(x_s), \qquad 0 \leq \alpha_s \leq C.$$

Substitute: The decision score now depends only on dot products:

$$f(x_t) = \sum_{s=1}^{n} \alpha_s y_s \underbrace{\langle \phi(x_s), \phi(x_t) \rangle}_{\text{dot product}} + b.$$

Nonlinear SVM

Transform: Fit a linear SVM using nonlinear features $\phi(x)$:

$$f(x_t) = \langle w, \phi(x_t) \rangle + b.$$

Why dot products? At the optimum, SVM weights are sums of transformed features:

$$w = \sum_{s=1}^{n} \alpha_s y_s \phi(x_s), \qquad 0 \leq \alpha_s \leq C.$$

Substitute: The decision score now depends only on dot products:

$$f(x_t) = \sum_{s=1}^{n} \alpha_s y_s \underbrace{\langle \phi(x_s), \phi(x_t) \rangle}_{\text{dot product}} + b.$$

Implication: To classify x_t, we need the dot products of its transformed features with those of the training observations.

Question: Can we compute these dot products without constructing $\phi(x)$?

C controls the soft-margin penalty and bounds the dual coefficients α_s.

The Kernel Trick: A Concrete Example

Suppose we know the transformation for $x = (x_1, x_2)^\top$:

$$\phi(x) = \begin{pmatrix} x_1^2 \\ \sqrt{2}\, x_1 x_2 \\ x_2^2 \end{pmatrix}.$$

The Kernel Trick: A Concrete Example

Suppose we know the transformation for $x = (x_1, x_2)^\top$:

$$\phi(x) = \begin{pmatrix} x_1^2 \\ \sqrt{2}\, x_1 x_2 \\ x_2^2 \end{pmatrix}.$$

The dot product we need can be simplified:

$$\begin{aligned} \phi(x_s)^\top \phi(x_t) &= x_{s1}^2 x_{t1}^2 + 2 x_{s1} x_{s2} x_{t1} x_{t2} + x_{s2}^2 x_{t2}^2 \\ &= \boxed{(x_s^\top x_t)^2} \equiv K(x_s, x_t). \end{aligned}$$

The Kernel Trick: A Concrete Example

Suppose we know the transformation for $x = (x_1, x_2)^\top$:

$$\phi(x) = \begin{pmatrix} x_1^2 \\ \sqrt{2}\, x_1 x_2 \\ x_2^2 \end{pmatrix}.$$

The dot product we need can be simplified:

$$\begin{aligned} \phi(x_s)^\top \phi(x_t) &= x_{s1}^2 x_{t1}^2 + 2 x_{s1} x_{s2} x_{t1} x_{t2} + x_{s2}^2 x_{t2}^2 \\ &= \boxed{(x_s^\top x_t)^2} \equiv K(x_s, x_t). \end{aligned}$$

This is a kernel: Calculate the *same* dot product directly from the original features, without constructing ϕ.

The Kernel Trick: A Concrete Example

Suppose we know the transformation for $x = (x_1, x_2)^\top$:

$$\phi(x) = \begin{pmatrix} x_1^2 \\ \sqrt{2}\, x_1 x_2 \\ x_2^2 \end{pmatrix}.$$

The dot product we need can be simplified:

$$\begin{aligned} \phi(x_s)^\top \phi(x_t) &= x_{s1}^2 x_{t1}^2 + 2 x_{s1} x_{s2} x_{t1} x_{t2} + x_{s2}^2 x_{t2}^2 \\ &= \boxed{(x_s^\top x_t)^2} \equiv K(x_s, x_t). \end{aligned}$$

This is a kernel: Calculate the *same* dot product directly from the original features, without constructing ϕ.

What if ϕ is much more complicated? Choose a valid kernel that implicitly defines a feature space. For RBF:

$$\boxed{K_\gamma(x_s, x_t) = \exp\left(-\gamma \|x_s - x_t\|^2\right)}.$$

Why exponential? Smooth similarity: $K = 1$ for identical observations and decreases with distance. Importantly, RBF is mathematically an exact inner product in *its own* (infinite-dimensional) feature space, not an approximation to some unknown ϕ.

RBF SVM: Putting It All Together

Optimization: Soft-margin SVM in the transformed feature space:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_{s=1}^{n}\xi_s \\ \text{s.t.} \quad & y_s\big(\langle w, \phi(x_s)\rangle + b\big) \geq 1 - \xi_s, \\ & \xi_s \geq 0. \end{aligned}$$

RBF SVM: Putting It All Together

Optimization: Soft-margin SVM in the transformed feature space:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_{s=1}^{n}\xi_s \\ \text{s.t.} \quad & y_s\big(\langle w, \phi(x_s)\rangle + b\big) \geq 1 - \xi_s, \\ & \xi_s \geq 0. \end{aligned}$$

Kernel trick: Instead of constructing $\phi(x)$, use

$$K_\gamma(x_s, x_t) = \langle \phi(x_s), \phi(x_t)\rangle = \exp\left(-\gamma\|x_s - x_t\|^2\right).$$

RBF SVM: Putting It All Together

Optimization: Soft-margin SVM in the transformed feature space:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_{s=1}^{n}\xi_s \\ \text{s.t.} \quad & y_s\big(\langle w, \phi(x_s)\rangle + b\big) \geq 1 - \xi_s, \\ & \xi_s \geq 0. \end{aligned}$$

Kernel trick: Instead of constructing $\phi(x)$, use

$$K_\gamma(x_s, x_t) = \langle \phi(x_s), \phi(x_t)\rangle = \exp\left(-\gamma\|x_s - x_t\|^2\right).$$

Resulting classification rule:

$$\hat{Y}_{t+1} = \text{sign}\left(\sum_{s\in SV} \alpha_s y_s K_\gamma(x_s, x_t) + b\right).$$

RBF SVM: Putting It All Together

Optimization: Soft-margin SVM in the transformed feature space:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_{s=1}^{n}\xi_s \\ \text{s.t.} \quad & y_s\big(\langle w, \phi(x_s)\rangle + b\big) \geq 1 - \xi_s, \\ & \xi_s \geq 0. \end{aligned}$$

Kernel trick: Instead of constructing $\phi(x)$, use

$$K_\gamma(x_s, x_t) = \langle \phi(x_s), \phi(x_t)\rangle = \exp\left(-\gamma\|x_s - x_t\|^2\right).$$

Resulting classification rule:

$$\hat{Y}_{t+1} = \text{sign}\left(\sum_{s\in SV} \alpha_s y_s K_\gamma(x_s, x_t) + b\right).$$

Hyperparameters:

- C: penalty for margin violations; larger C permits less regularization.
- γ: controls the locality of similarity; larger γ permits more flexible boundaries.
- Scale features and choose (C, γ) using CV.

Outline

1. Introduction

2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
   - Goodness of Fit

3. Multi Class Classification

Confusion Matrix

Q: Can you interpret each of the metrics?

Confusion Matrix

Q: Can you interpret each of the metrics?

	$\widehat{Y}_{t+1} = 1$ (Predict Bear)	$\widehat{Y}_{t+1} = 0$ (Predict Not-Bear)
$Y_{t+1} = 1$ (Bear)	True Positive (TP)	False Negative (FN)
$Y_{t+1} = 0$ (Not-Bear)	False Positive (FP)	True Negative (TN)

Each cell is the number (or frequency) of observations in the evaluation sample.

- $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$
- $Precision = \frac{TP}{TP+FP}$
- $Recall = \frac{TP}{TP+FN}$
- $Specificity = \frac{TN}{TN+FP}$
- Type I Error $= \frac{FP}{FP+TN}$
- Type II Error $= \frac{FN}{TP+FN}$

Confusion Matrix

Q: Can you interpret each of the metrics?

	$\widehat{Y}_{t+1} = 1$ (Predict Bear)	$\widehat{Y}_{t+1} = 0$ (Predict Not-Bear)
$Y_{t+1} = 1$ (Bear)	True Positive (TP)	False Negative (FN)
$Y_{t+1} = 0$ (Not-Bear)	False Positive (FP)	True Negative (TN)

Each cell is the number (or frequency) of observations in the evaluation sample.

- $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$ (Fraction of periods where the regime was correctly classified)
- $Precision = \frac{TP}{TP+FP}$
- $Recall = \frac{TP}{TP+FN}$
- $Specificity = \frac{TN}{TN+FP}$
- Type I Error $= \frac{FP}{FP+TN}$
- Type II Error $= \frac{FN}{TP+FN}$

Confusion Matrix

Q: Can you interpret each of the metrics?

	$\widehat{Y}_{t+1} = 1$ (Predict Bear)	$\widehat{Y}_{t+1} = 0$ (Predict Not-Bear)
$Y_{t+1} = 1$ (Bear)	True Positive (TP)	False Negative (FN)
$Y_{t+1} = 0$ (Not-Bear)	False Positive (FP)	True Negative (TN)

Each cell is the number (or frequency) of observations in the evaluation sample.

- $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$ (Fraction of periods where the regime was correctly classified)
- $Precision = \frac{TP}{TP+FP}$ (Among periods predicted Bear, the fraction that were actually Bear)
- $Recall = \frac{TP}{TP+FN}$
- $Specificity = \frac{TN}{TN+FP}$
- Type I Error $= \frac{FP}{FP+TN}$
- Type II Error $= \frac{FN}{TP+FN}$

Confusion Matrix

Q: Can you interpret each of the metrics?

	$\widehat{Y}_{t+1} = 1$ (Predict Bear)	$\widehat{Y}_{t+1} = 0$ (Predict Not-Bear)
$Y_{t+1} = 1$ (Bear)	True Positive (TP)	False Negative (FN)
$Y_{t+1} = 0$ (Not-Bear)	False Positive (FP)	True Negative (TN)

Each cell is the number (or frequency) of observations in the evaluation sample.

- $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$ (Fraction of periods where the regime was correctly classified)
- $Precision = \frac{TP}{TP+FP}$ (Among periods predicted Bear, the fraction that were actually Bear)
- $Recall = \frac{TP}{TP+FN}$ (Among all Bear periods, the fraction correctly identified)
- $Specificity = \frac{TN}{TN+FP}$
- Type I Error $= \frac{FP}{FP+TN}$
- Type II Error $= \frac{FN}{TP+FN}$

Confusion Matrix

Q: Can you interpret each of the metrics?

	$\widehat{Y}_{t+1} = 1$ (Predict Bear)	$\widehat{Y}_{t+1} = 0$ (Predict Not-Bear)
$Y_{t+1} = 1$ (Bear)	True Positive (TP)	False Negative (FN)
$Y_{t+1} = 0$ (Not-Bear)	False Positive (FP)	True Negative (TN)

Each cell is the number (or frequency) of observations in the evaluation sample.

- $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$ (Fraction of periods where the regime was correctly classified)
- $Precision = \frac{TP}{TP+FP}$ (Among periods predicted Bear, the fraction that were actually Bear)
- $Recall = \frac{TP}{TP+FN}$ (Among all Bear periods, the fraction correctly identified)
- $Specificity = \frac{TN}{TN+FP}$ (Among Not-Bear periods, the fraction correctly identified)
- Type I Error $= \frac{FP}{FP+TN}$
- Type II Error $= \frac{FN}{TP+FN}$

Confusion Matrix

Q: Can you interpret each of the metrics?

	$\widehat{Y}_{t+1} = 1$ (Predict Bear)	$\widehat{Y}_{t+1} = 0$ (Predict Not-Bear)
$Y_{t+1} = 1$ (Bear)	True Positive (TP)	False Negative (FN)
$Y_{t+1} = 0$ (Not-Bear)	False Positive (FP)	True Negative (TN)

Each cell is the number (or frequency) of observations in the evaluation sample.

- $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$ (Fraction of periods where the regime was correctly classified)
- $Precision = \frac{TP}{TP+FP}$ (Among periods predicted Bear, the fraction that were actually Bear)
- $Recall = \frac{TP}{TP+FN}$ (Among all Bear periods, the fraction correctly identified)
- $Specificity = \frac{TN}{TN+FP}$ (Among Not-Bear periods, the fraction correctly identified)
- Type I Error $= \frac{FP}{FP+TN}$ (False alarm rate aka False positive rate: predicting Bear when it was Not-Bear)
- Type II Error $= \frac{FN}{TP+FN}$

Confusion Matrix

Q: Can you interpret each of the metrics?

	$\widehat{Y}_{t+1} = 1$ (Predict Bear)	$\widehat{Y}_{t+1} = 0$ (Predict Not-Bear)
$Y_{t+1} = 1$ (Bear)	True Positive (TP)	False Negative (FN)
$Y_{t+1} = 0$ (Not-Bear)	False Positive (FP)	True Negative (TN)

Each cell is the number (or frequency) of observations in the evaluation sample.

- $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$ (Fraction of periods where the regime was correctly classified)
- $Precision = \frac{TP}{TP+FP}$ (Among periods predicted Bear, the fraction that were actually Bear)
- $Recall = \frac{TP}{TP+FN}$ (Among all Bear periods, the fraction correctly identified)
- $Specificity = \frac{TN}{TN+FP}$ (Among Not-Bear periods, the fraction correctly identified)
- Type I Error $= \frac{FP}{FP+TN}$ (False alarm rate aka False positive rate: predicting Bear when it was Not-Bear)
- Type II Error $= \frac{FN}{TP+FN}$ (Missed crisis rate: failing to predict Bear)

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP =$

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1)$

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN =$

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN = Pr(\hat{Y} = 0 \mid Y = 1)$

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN = Pr(\hat{Y} = 0 \mid Y = 1) = 1$ (misses every Bear)

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN = Pr(\hat{Y} = 0 \mid Y = 1) = 1$ (misses every Bear)
- $FP =$

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN = Pr(\hat{Y} = 0 \mid Y = 1) = 1$ (misses every Bear)
- $FP = Pr(\hat{Y} = 1 \mid Y = 0)$

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN = Pr(\hat{Y} = 0 \mid Y = 1) = 1$ (misses every Bear)
- $FP = Pr(\hat{Y} = 1 \mid Y = 0) = 0$ (never raises a false alarm)

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN = Pr(\hat{Y} = 0 \mid Y = 1) = 1$ (misses every Bear)
- $FP = Pr(\hat{Y} = 1 \mid Y = 0) = 0$ (never raises a false alarm)
- $TN =$

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN = Pr(\hat{Y} = 0 \mid Y = 1) = 1$ (misses every Bear)
- $FP = Pr(\hat{Y} = 1 \mid Y = 0) = 0$ (never raises a false alarm)
- $TN = Pr(\hat{Y} = 0 \mid Y = 0)$

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN = Pr(\hat{Y} = 0 \mid Y = 1) = 1$ (misses every Bear)
- $FP = Pr(\hat{Y} = 1 \mid Y = 0) = 0$ (never raises a false alarm)
- $TN = Pr(\hat{Y} = 0 \mid Y = 0) = 1$ (always correctly predicts Not-Bear)

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN = Pr(\hat{Y} = 0 \mid Y = 1) = 1$ (misses every Bear)
- $FP = Pr(\hat{Y} = 1 \mid Y = 0) = 0$ (never raises a false alarm)
- $TN = Pr(\hat{Y} = 0 \mid Y = 0) = 1$ (always correctly predicts Not-Bear)

Q: What is accuracy in this model?

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN = Pr(\hat{Y} = 0 \mid Y = 1) = 1$ (misses every Bear)
- $FP = Pr(\hat{Y} = 1 \mid Y = 0) = 0$ (never raises a false alarm)
- $TN = Pr(\hat{Y} = 0 \mid Y = 0) = 1$ (always correctly predicts Not-Bear)

Q: What is accuracy in this model?

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{TN}{TN + FN} = \frac{90}{90 + 10} \approx 90\%$$

Why Accuracy Can Be Misleading

Imbalanced classes in Bear market prediction

Suppose Bear markets are rare: $\mathbb{P}(Y_{t+1} = 1) \ll \mathbb{P}(Y_{t+1} = 0)$ (e.g., 10% Bear, 90% Not-Bear).

Consider a naive classifier:

$$\widehat{Y}_{t+1} = 0 \quad \text{for all } t$$

(i.e., always predict Not-Bear).

Q: Can you fill in the elements of a confusion matrix?

- $TP = Pr(\hat{Y} = 1 \mid Y = 1) = 0$ (never correctly predicts Bear)
- $FN = Pr(\hat{Y} = 0 \mid Y = 1) = 1$ (misses every Bear)
- $FP = Pr(\hat{Y} = 1 \mid Y = 0) = 0$ (never raises a false alarm)
- $TN = Pr(\hat{Y} = 0 \mid Y = 0) = 1$ (always correctly predicts Not-Bear)

Q: What is accuracy in this model?

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{TN}{TN + FN} = \frac{90}{90 + 10} \approx 90\%$$

Conclusion: with rare Bear markets, accuracy can be dominated by TN and can reward useless models.

F1 Score

F1 Score

When the positive class (Bear) is rare, we care about both:

- **Precision:** do we avoid crying wolf?

$$\text{Precision} = \frac{TP}{TP + FP}$$

F1 Score

When the positive class (Bear) is rare, we care about both:

- **Precision:** do we avoid crying wolf?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** do we catch crises?

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score

When the positive class (Bear) is rare, we care about both:

- **Precision:** do we avoid crying wolf?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** do we catch crises?

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score (harmonic mean):

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1 Score

When the positive class (Bear) is rare, we care about both:

- **Precision:** do we avoid crying wolf?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** do we catch crises?

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score (harmonic mean):

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Penalizes models that have high precision but low recall (miss crises), or high recall but low precision (constant false alarms).

F1 Score

When the positive class (Bear) is rare, we care about both:

- **Precision:** do we avoid crying wolf?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** do we catch crises?

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score (harmonic mean):

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- Penalizes models that have high precision but low recall (miss crises), or high recall but low precision (constant false alarms).
- Helpful in rare-event settings because it emphasizes performance on the Bear class (it does not "get credit" for many easy true negatives).

ROC Curve and AUC

Ranking Quality Rather Than Classification Accuracy

ROC Curve and AUC

Ranking Quality Rather Than Classification Accuracy

So far, evaluation required choosing a threshold τ:

ROC Curve and AUC

Ranking Quality Rather Than Classification Accuracy

So far, evaluation required choosing a threshold τ: $\widehat{Y}_{t+1} = \mathbf{1}\{\hat{p}_{t+1} \geq \tau\}$

ROC Curve and AUC

Ranking Quality Rather Than Classification Accuracy

So far, evaluation required choosing a threshold τ: $\widehat{Y}_{t+1} = \mathbf{1}\{\hat{p}_{t+1} \geq \tau\}$

ROC Curve:

- For each threshold τ, compute:

$$\text{TPR}(\tau) = \frac{TP}{TP + FN}, \qquad \text{FPR}(\tau) = \frac{FP}{FP + TN}$$

ROC Curve and AUC

Ranking Quality Rather Than Classification Accuracy

So far, evaluation required choosing a threshold τ: $\widehat{Y}_{t+1} = \mathbf{1}\{\hat{p}_{t+1} \geq \tau\}$

ROC Curve:

- For each threshold τ, compute:

$$\text{TPR}(\tau) = \frac{TP}{TP + FN}, \qquad \text{FPR}(\tau) = \frac{FP}{FP + TN}$$

- Plot TPR(τ) vs FPR(τ) as τ varies.

ROC Curve and AUC

Ranking Quality Rather Than Classification Accuracy

So far, evaluation required choosing a threshold τ: $\widehat{Y}_{t+1} = \mathbf{1}\{\hat{p}_{t+1} \geq \tau\}$

ROC Curve:

- For each threshold τ, compute:

$$\text{TPR}(\tau) = \frac{TP}{TP + FN}, \qquad \text{FPR}(\tau) = \frac{FP}{FP + TN}$$

- Plot TPR(τ) vs FPR(τ) as τ varies.

Area Under the Curve (AUC):

$$AUC = \mathbb{P}\big(\hat{p}_{Bear} > \hat{p}_{Not\text{-}Bear}\big)$$

ROC Curve and AUC

Ranking Quality Rather Than Classification Accuracy

So far, evaluation required choosing a threshold τ: $\widehat{Y}_{t+1} = \mathbf{1}\{\hat{p}_{t+1} \geq \tau\}$

ROC Curve:

- For each threshold τ, compute:

$$\text{TPR}(\tau) = \frac{TP}{TP + FN}, \qquad \text{FPR}(\tau) = \frac{FP}{FP + TN}$$

- Plot TPR(τ) vs FPR(τ) as τ varies.

Area Under the Curve (AUC):

$$AUC = \mathbb{P}\big(\hat{p}_{Bear} > \hat{p}_{Not\text{-}Bear}\big)$$

- Probability that a randomly chosen Bear period receives a higher predicted score than a randomly chosen Not-Bear period.

ROC Curve and AUC

Ranking Quality Rather Than Classification Accuracy

So far, evaluation required choosing a threshold τ: $\widehat{Y}_{t+1} = \mathbf{1}\{\hat{p}_{t+1} \geq \tau\}$

ROC Curve:

- For each threshold τ, compute:

$$\text{TPR}(\tau) = \frac{TP}{TP + FN}, \qquad \text{FPR}(\tau) = \frac{FP}{FP + TN}$$

- Plot TPR(τ) vs FPR(τ) as τ varies.

Area Under the Curve (AUC):

$$AUC = \mathbb{P}\big(\hat{p}_{Bear} > \hat{p}_{Not\text{-}Bear}\big)$$

- Probability that a randomly chosen Bear period receives a higher predicted score than a randomly chosen Not-Bear period.
- Evaluates the model as a **ranker**, not a classifier.

ROC Curve and AUC

Ranking Quality Rather Than Classification Accuracy

So far, evaluation required choosing a threshold τ: $\widehat{Y}_{t+1} = \mathbf{1}\{\hat{p}_{t+1} \geq \tau\}$

ROC Curve:

- For each threshold τ, compute:

$$\text{TPR}(\tau) = \frac{TP}{TP + FN}, \qquad \text{FPR}(\tau) = \frac{FP}{FP + TN}$$

- Plot TPR(τ) vs FPR(τ) as τ varies.

Area Under the Curve (AUC):

$$AUC = \mathbb{P}\big(\hat{p}_{Bear} > \hat{p}_{Not\text{-}Bear}\big)$$

- Probability that a randomly chosen Bear period receives a higher predicted score than a randomly chosen Not-Bear period.
- Evaluates the model as a **ranker**, not a classifier.
- $AUC = 0.5$: random ordering $\qquad$ $AUC = 1$: perfect separation.

ROC Curve and AUC

Ranking Quality Rather Than Classification Accuracy

So far, evaluation required choosing a threshold τ: $\widehat{Y}_{t+1} = \mathbf{1}\{\hat{p}_{t+1} \geq \tau\}$

ROC Curve:

- For each threshold τ, compute:

$$\text{TPR}(\tau) = \frac{TP}{TP + FN}, \qquad \text{FPR}(\tau) = \frac{FP}{FP + TN}$$

- Plot TPR(τ) vs FPR(τ) as τ varies.

Area Under the Curve (AUC):

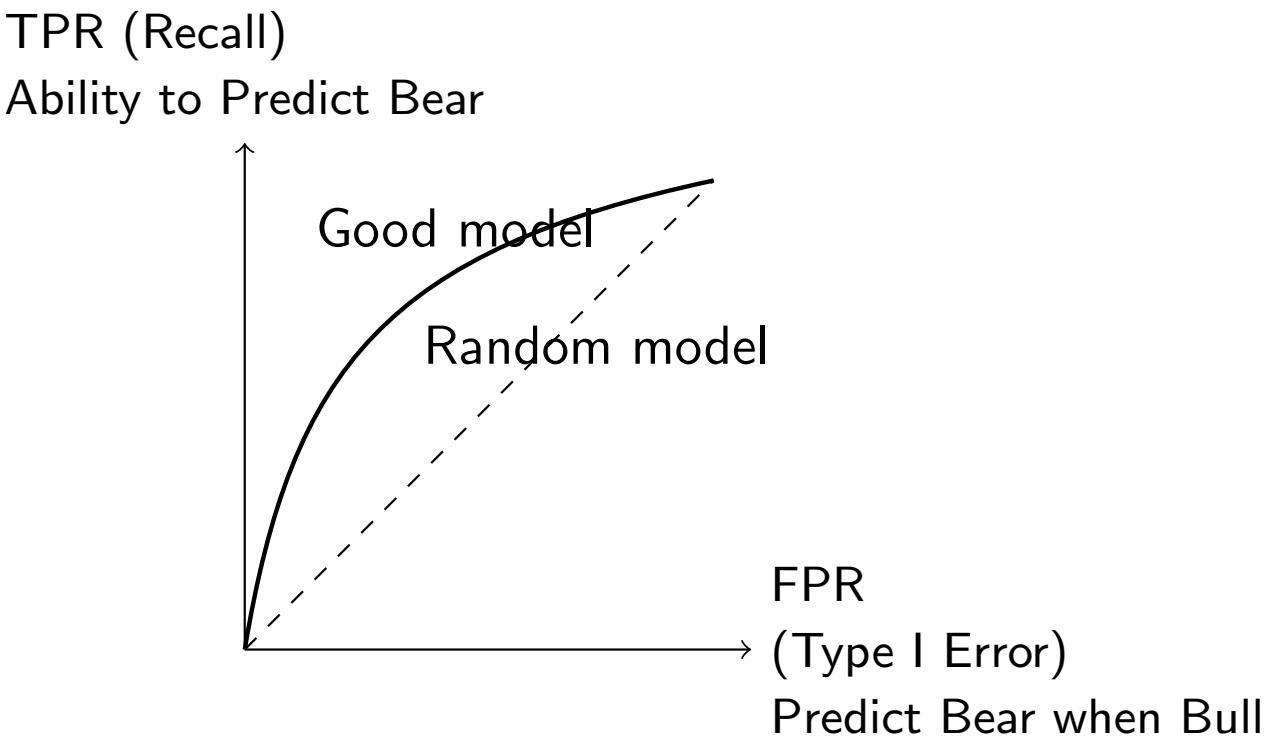
$$AUC = \mathbb{P}\big(\hat{p}_{Bear} > \hat{p}_{Not\text{-}Bear}\big)$$

- Probability that a randomly chosen Bear period receives a higher predicted score than a randomly chosen Not-Bear period.
- Evaluates the model as a **ranker**, not a classifier.
- $AUC = 0.5$: random ordering $\qquad$ $AUC = 1$: perfect separation.

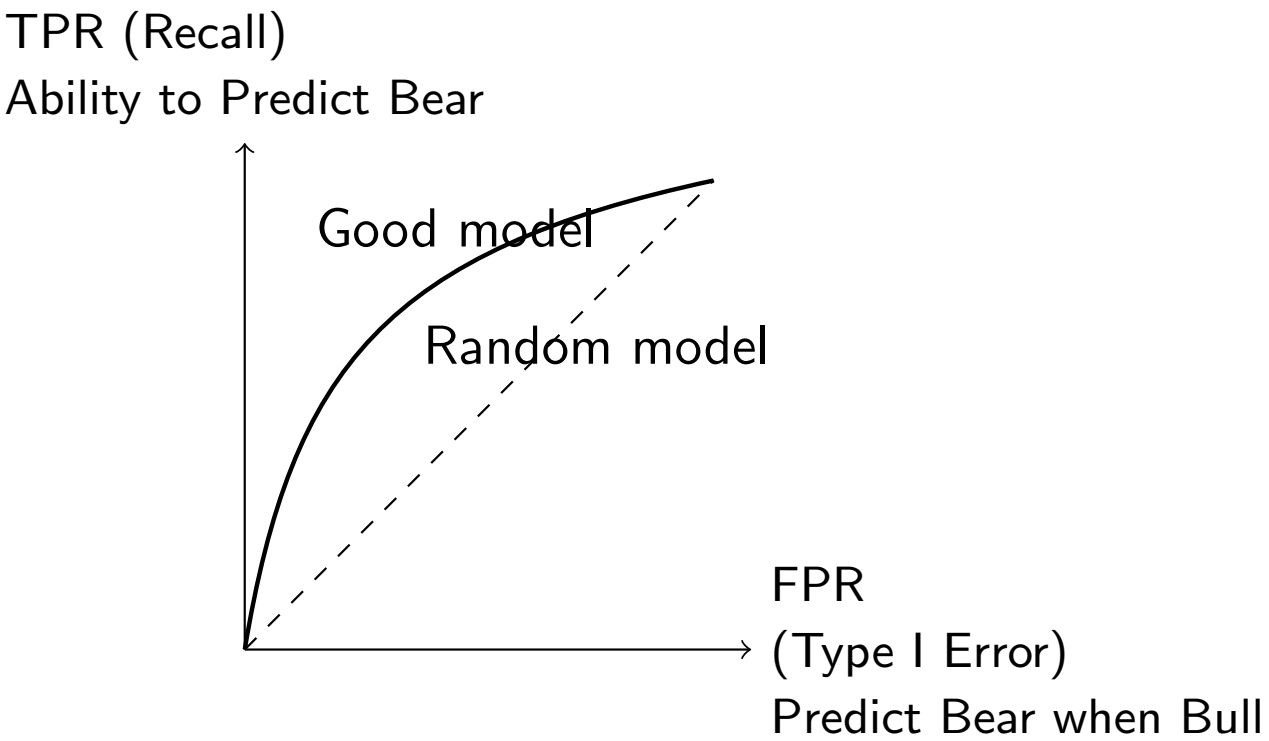
Interpretation: Even if the cutoff changes, a high-AUC model consistently places crisis periods above normal periods in predicted risk.

ROC Curve Illustration

ROC Curve Illustration

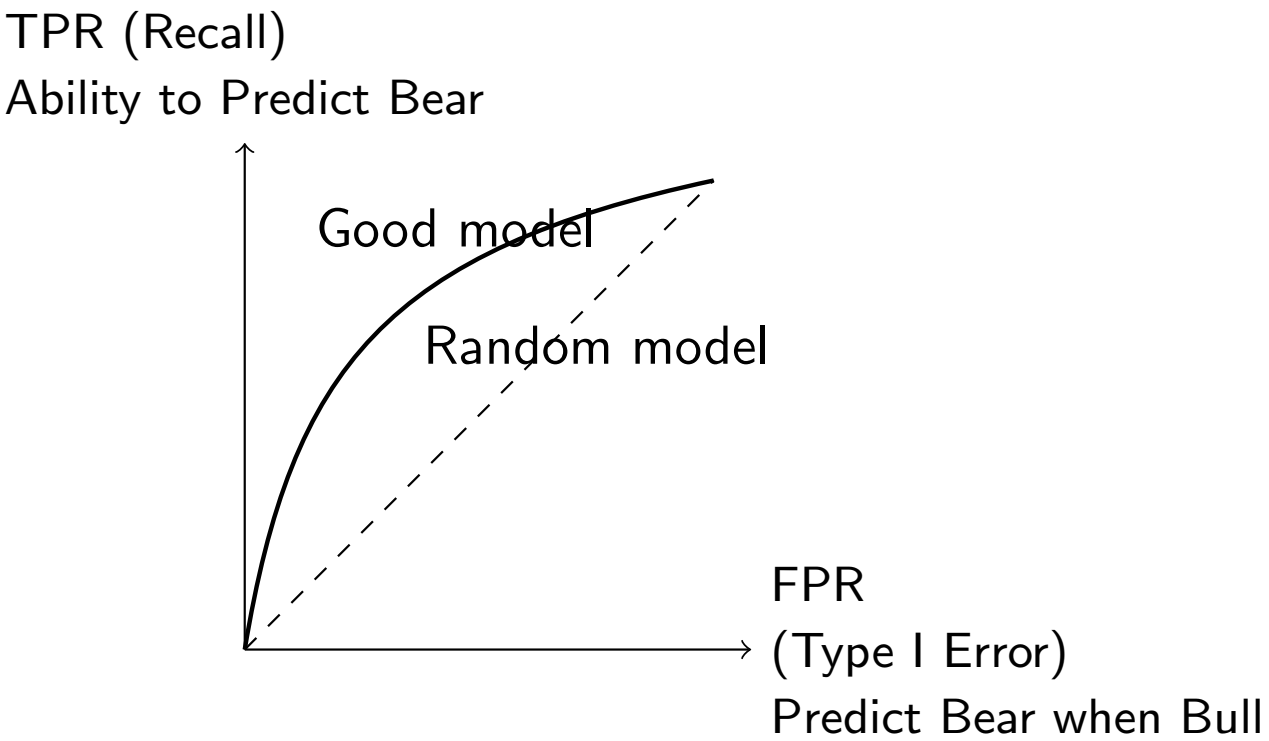


ROC Curve Illustration



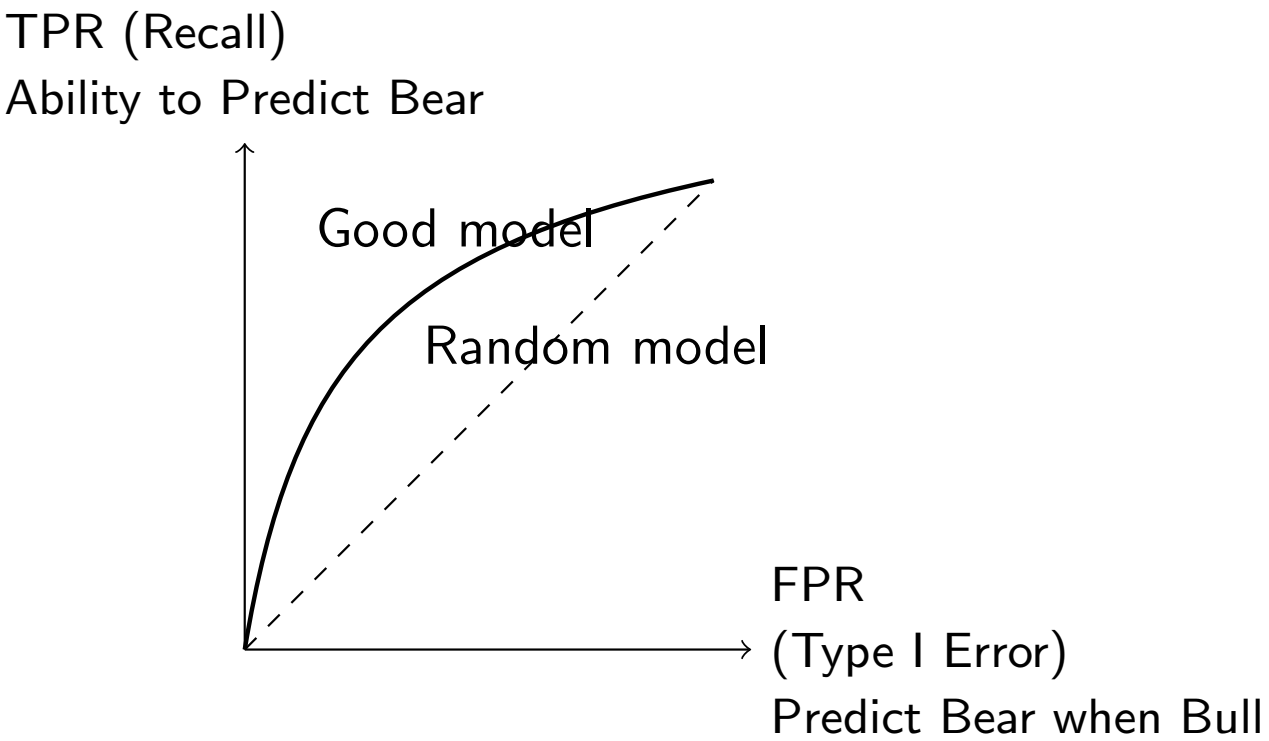
- Each point corresponds to a different probability threshold τ.

ROC Curve Illustration



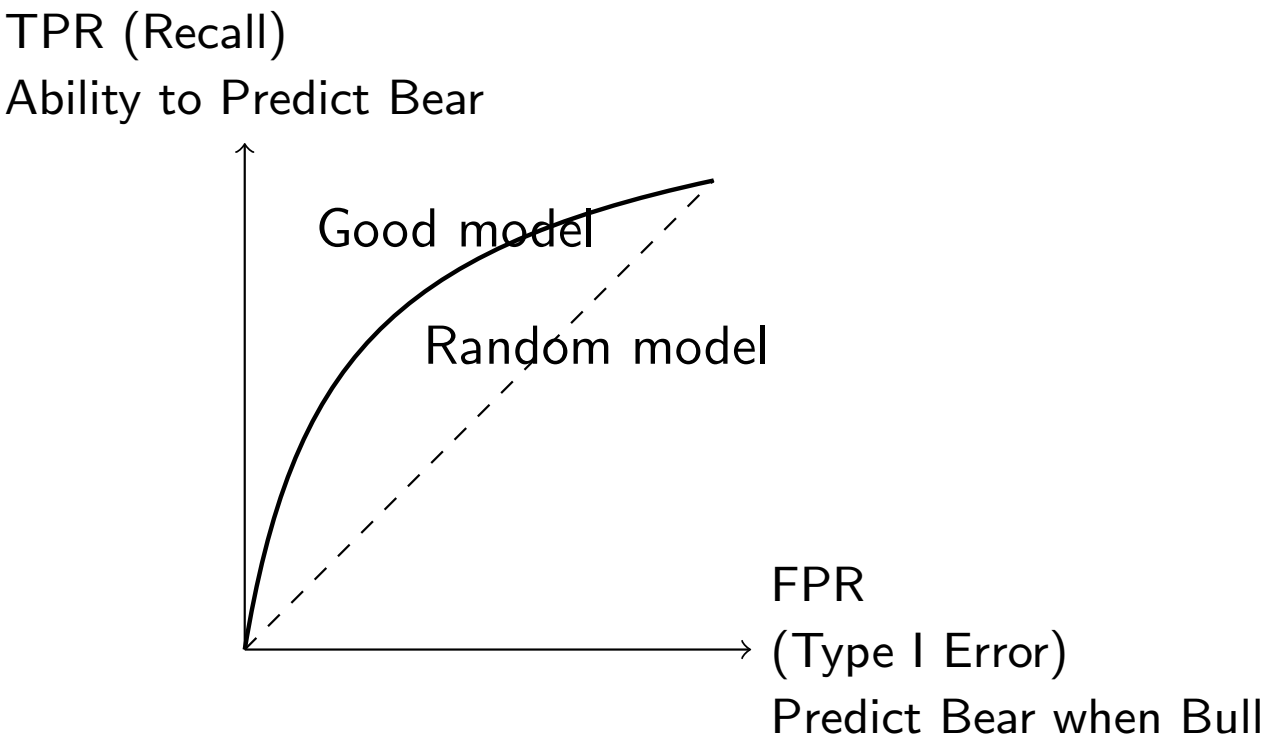
- Each point corresponds to a different probability threshold τ.
- The more the ROC curve bows toward the upper-left, the

ROC Curve Illustration



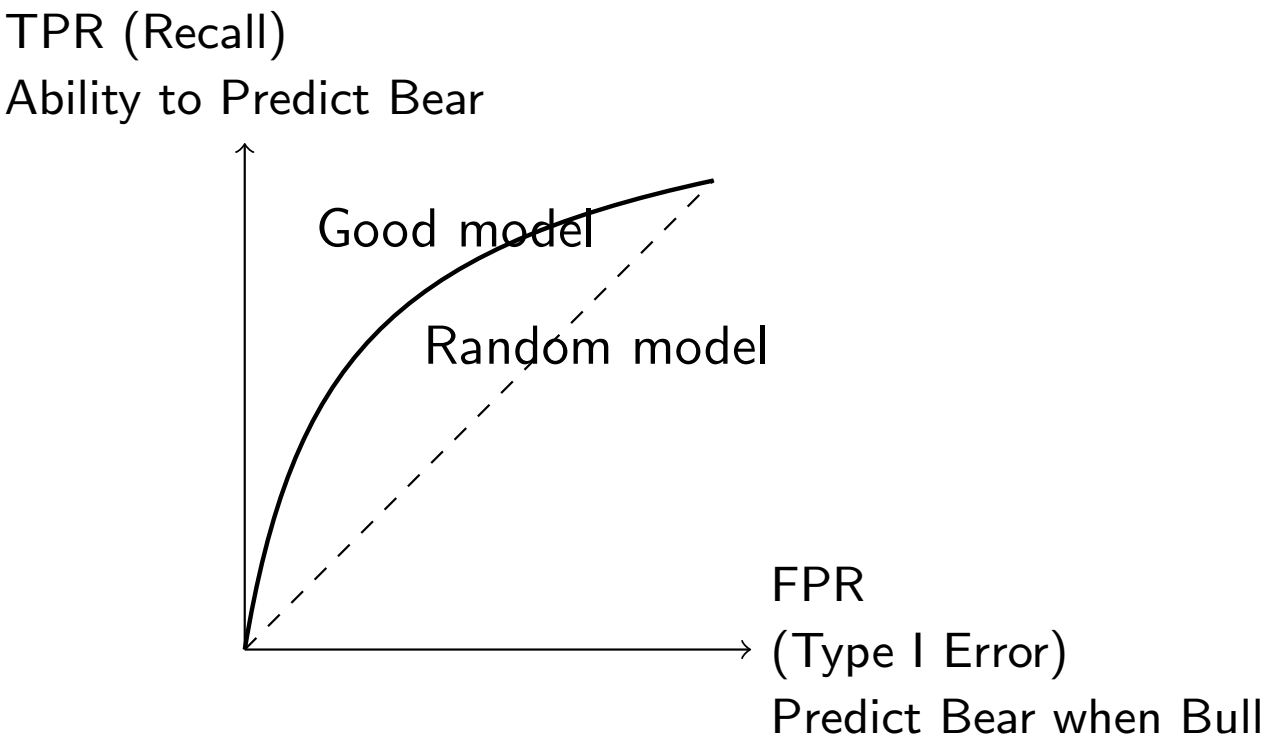
- Each point corresponds to a different probability threshold τ.
- The more the ROC curve bows toward the upper-left, the stronger the separation between Bear and Not-Bear periods.

ROC Curve Illustration



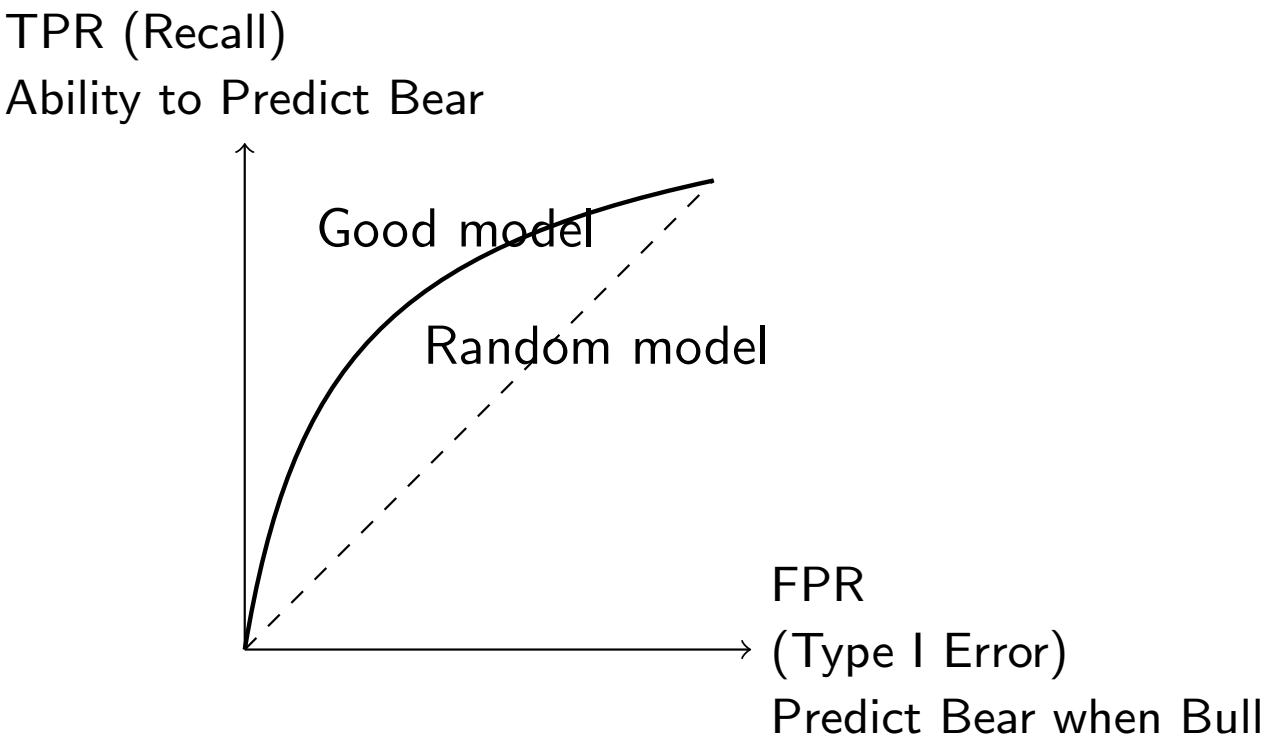
- Each point corresponds to a different probability threshold τ.
- The more the ROC curve bows toward the upper-left, the stronger the separation between Bear and Not-Bear periods.
- $AUC = 0.5$ corresponds to

ROC Curve Illustration



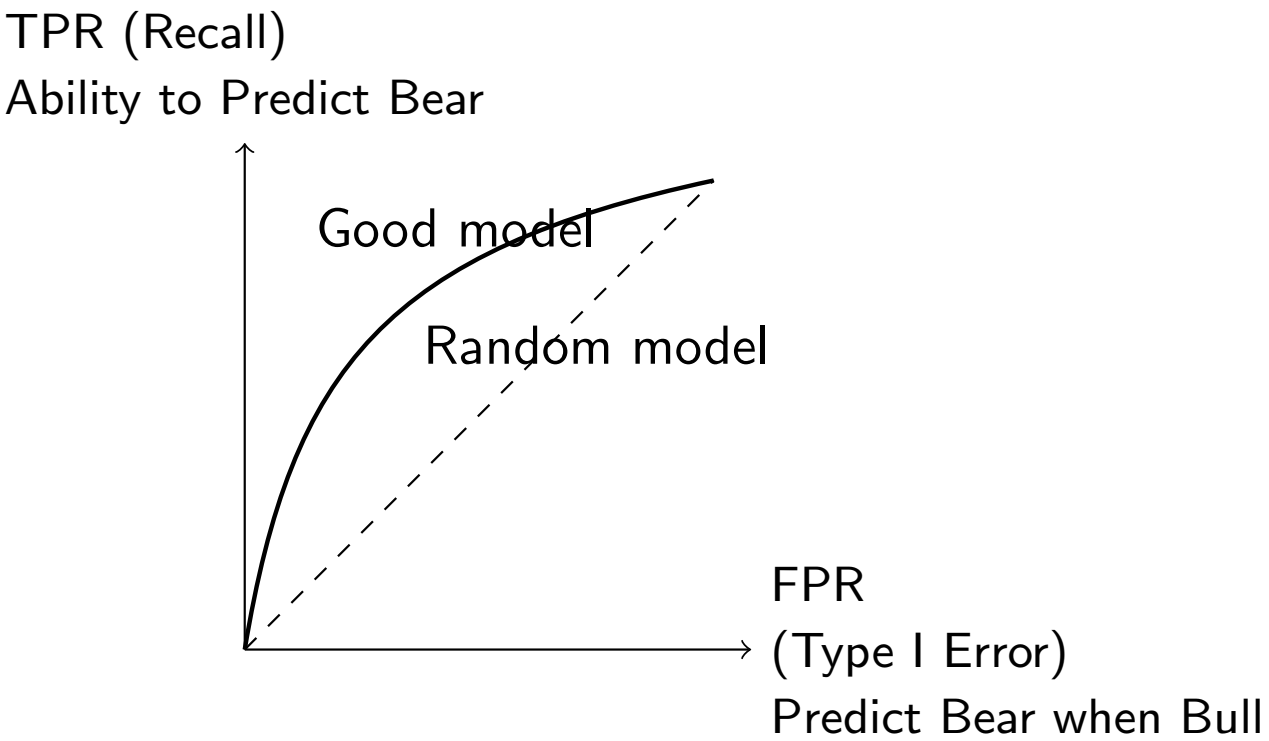
- Each point corresponds to a different probability threshold τ.
- The more the ROC curve bows toward the upper-left, the stronger the separation between Bear and Not-Bear periods.
- $AUC = 0.5$ corresponds to random ordering.

ROC Curve Illustration



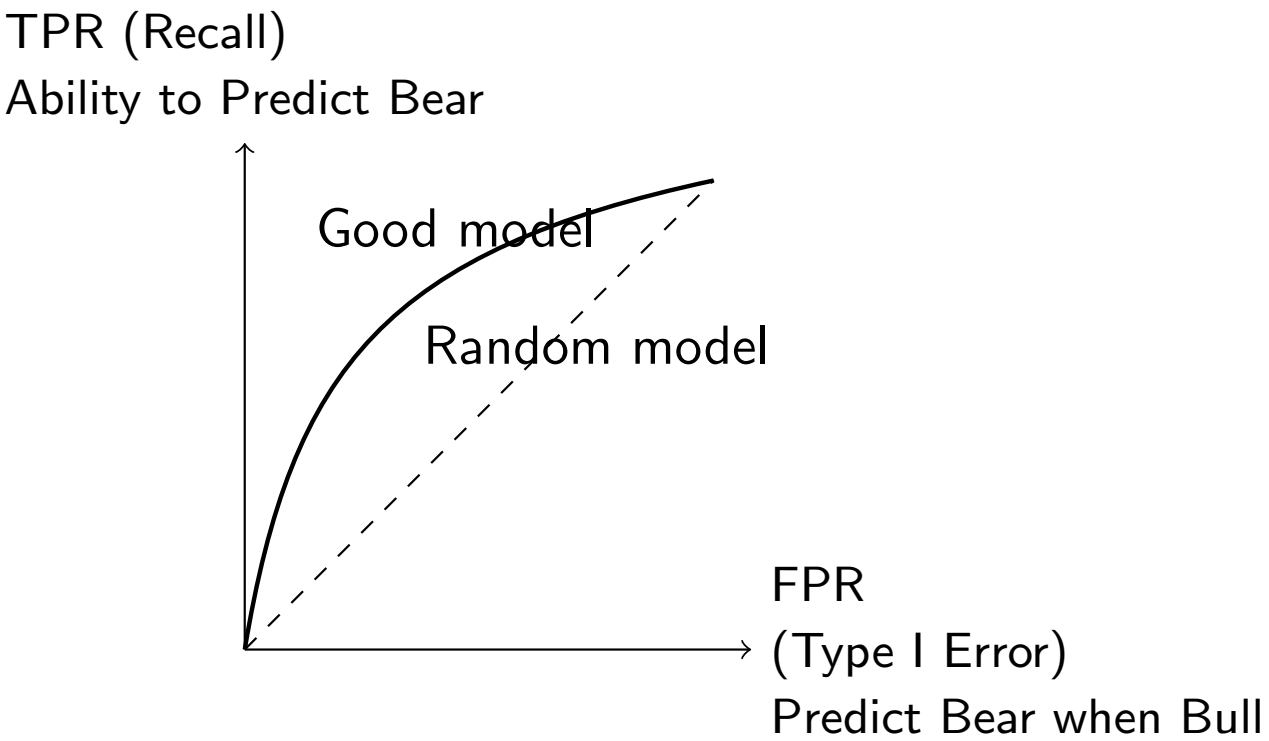
- Each point corresponds to a different probability threshold τ.
- The more the ROC curve bows toward the upper-left, the stronger the separation between Bear and Not-Bear periods.
- $AUC = 0.5$ corresponds to random ordering.
- $AUC = 1$ corresponds to

ROC Curve Illustration



- Each point corresponds to a different probability threshold τ.
- The more the ROC curve bows toward the upper-left, the stronger the separation between Bear and Not-Bear periods.
- $AUC = 0.5$ corresponds to random ordering.
- $AUC = 1$ corresponds to perfect ranking.

ROC Curve Illustration



- Each point corresponds to a different probability threshold τ.
- The more the ROC curve bows toward the upper-left, the stronger the separation between Bear and Not-Bear periods.
- $AUC = 0.5$ corresponds to random ordering.
- $AUC = 1$ corresponds to perfect ranking.

Interpretation: A high-AUC model improves crisis detection (vertical axis) without generating excessive false alarms (horizontal axis).

How Do We Choose the Threshold τ?

We predict Bear if: $\hat{p}(X) \geq \tau$

Decision-theoretic approach: choose the action that minimizes expected loss.

How Do We Choose the Threshold τ?

We predict Bear if: $\hat{p}(X) \geq \tau$

Decision-theoretic approach: choose the action that minimizes expected loss. Define the costs:

- C_{FP} = cost of false alarm (predict Bear when Bull)
- C_{FN} = cost of missed crisis (predict Bull when Bear)

How Do We Choose the Threshold τ?

We predict Bear if: $\hat{p}(X) \geq \tau$

Decision-theoretic approach: choose the action that minimizes expected loss. Define the costs:

- C_{FP} = cost of false alarm (predict Bear when Bull)
- C_{FN} = cost of missed crisis (predict Bull when Bear)

Let the model estimate

$$p(X) = P(Y = 1 \mid X)$$

How Do We Choose the Threshold τ?

We predict Bear if: $\hat{p}(X) \geq \tau$

Decision-theoretic approach: choose the action that minimizes expected loss. Define the costs:

- C_{FP} = cost of false alarm (predict Bear when Bull)
- C_{FN} = cost of missed crisis (predict Bull when Bear)

Let the model estimate

$$p(X) = P(Y = 1 \mid X)$$

Expected loss conditional on X:

$$L(\text{Bear} \mid X) = C_{FP}(1 - p(X))$$

$$L(\text{Not-Bear} \mid X) = C_{FN}p(X)$$

How Do We Choose the Threshold τ?

We predict Bear if: $\hat{p}(X) \geq \tau$

Decision-theoretic approach: choose the action that minimizes expected loss. Define the costs:

- C_{FP} = cost of false alarm (predict Bear when Bull)
- C_{FN} = cost of missed crisis (predict Bull when Bear)

Let the model estimate

$$p(X) = P(Y = 1 \mid X)$$

Expected loss conditional on X:

$$L(\text{Bear} \mid X) = C_{FP}(1 - p(X))$$

$$L(\text{Not-Bear} \mid X) = C_{FN} p(X)$$

Choose Bear when

$$C_{FP}(1 - p(X)) < C_{FN} p(X)$$

How Do We Choose the Threshold τ? cont'd

which implies the optimal threshold

$$\tau^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

How Do We Choose the Threshold τ? cont'd

which implies the optimal threshold

$$\tau^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

- If missed crises are very costly $\Rightarrow$ choose low τ.
- If false alarms are very costly $\Rightarrow$ choose high τ.

How Do We Choose the Threshold τ? cont'd

which implies the optimal threshold

$$\tau^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$$

- If missed crises are very costly $\Rightarrow$ choose low τ.
- If false alarms are very costly $\Rightarrow$ choose high τ.

Important: the costs C_{FP} and C_{FN} reflect user preferences and are chosen based on the economic context.

Brier Score

Evaluating Probability Predictions

Brier Score

Evaluating Probability Predictions

For probabilistic forecasts $\hat{p}_{t+1}$:

$$\text{Brier Score} = \frac{1}{T}\sum_{t=1}^{T}(Y_{t+1} - \hat{p}_{t+1})^2$$

Brier Score

Evaluating Probability Predictions

For probabilistic forecasts $\hat{p}_{t+1}$:

$$\text{Brier Score} = \frac{1}{T}\sum_{t=1}^{T}(Y_{t+1} - \hat{p}_{t+1})^2$$

- Mean squared error of probability forecasts.

Brier Score

Evaluating Probability Predictions

For probabilistic forecasts $\hat{p}_{t+1}$:

$$\text{Brier Score} = \frac{1}{T}\sum_{t=1}^{T}(Y_{t+1} - \hat{p}_{t+1})^2$$

- Mean squared error of probability forecasts.
- Proper scoring rule: best forecast is the true conditional probability.

Brier Score

Evaluating Probability Predictions

For probabilistic forecasts $\hat{p}_{t+1}$:

$$\text{Brier Score} = \frac{1}{T}\sum_{t=1}^{T}(Y_{t+1} - \hat{p}_{t+1})^2$$

- Mean squared error of probability forecasts.
- Proper scoring rule: best forecast is the true conditional probability.
- Penalizes overconfident wrong forecasts.

Brier Score

Evaluating Probability Predictions

For probabilistic forecasts $\hat{p}_{t+1}$:

$$\text{Brier Score} = \frac{1}{T}\sum_{t=1}^{T}(Y_{t+1} - \hat{p}_{t+1})^2$$

- Mean squared error of probability forecasts.
- Proper scoring rule: best forecast is the true conditional probability.
- Penalizes overconfident wrong forecasts.

Interpretation examples:

- Predict 0.9 probability of Bear and no Bear occurs $\rightarrow$ large penalty.

Brier Score

Evaluating Probability Predictions

For probabilistic forecasts $\hat{p}_{t+1}$:

$$\text{Brier Score} = \frac{1}{T}\sum_{t=1}^{T}(Y_{t+1} - \hat{p}_{t+1})^2$$

- Mean squared error of probability forecasts.
- Proper scoring rule: best forecast is the true conditional probability.
- Penalizes overconfident wrong forecasts.

Interpretation examples:

- Predict 0.9 probability of Bear and no Bear occurs $\rightarrow$ large penalty.
- Predict 0.6 probability of Bear and Bear occurs $\rightarrow$ small penalty.

Brier Score

Evaluating Probability Predictions

For probabilistic forecasts $\hat{p}_{t+1}$:

$$\text{Brier Score} = \frac{1}{T}\sum_{t=1}^{T}(Y_{t+1} - \hat{p}_{t+1})^2$$

- Mean squared error of probability forecasts.
- Proper scoring rule: best forecast is the true conditional probability.
- Penalizes overconfident wrong forecasts.

Interpretation examples:

- Predict 0.9 probability of Bear and no Bear occurs $\rightarrow$ large penalty.
- Predict 0.6 probability of Bear and Bear occurs $\rightarrow$ small penalty.

Interpretation: Calibration matters. Overconfident crisis forecasts can be economically costly.

Outline

1. Introduction
2. Binary Classification
3. Multi Class Classification
   - k-Nearest Neighbors (k-NN)
   - CART - Classification and Regression Tree

Why multi-class?

Why multi-class?

Binary regimes can be too coarse:

- "neutral" or "transition" states are common in practice
- decision rules often differ across three states (risk-on / neutral / risk-off)

Why multi-class?

Binary regimes can be too coarse:

- "neutral" or "transition" states are common in practice
- decision rules often differ across three states (risk-on / neutral / risk-off)

Let $Y_t \in \{1, 2, 3\}$ denote (bull, neutral, bear), with features X_t.

Why multi-class?

Binary regimes can be too coarse:

- "neutral" or "transition" states are common in practice
- decision rules often differ across three states (risk-on / neutral / risk-off)

Let $Y_t \in \{1, 2, 3\}$ denote (bull, neutral, bear), with features X_t.

We can extend probabilistic and margin-based methods to multi-class settings:

- Multinomial logit (softmax probabilities)

Why multi-class?

Binary regimes can be too coarse:

- "neutral" or "transition" states are common in practice
- decision rules often differ across three states (risk-on / neutral / risk-off)

Let $Y_t \in \{1, 2, 3\}$ denote (bull, neutral, bear), with features X_t.

We can extend probabilistic and margin-based methods to multi-class settings:

- Multinomial logit (softmax probabilities)
- One-vs-rest or one-vs-one SVM

Why multi-class?

Binary regimes can be too coarse:

- "neutral" or "transition" states are common in practice
- decision rules often differ across three states (risk-on / neutral / risk-off)

Let $Y_t \in \{1, 2, 3\}$ denote (bull, neutral, bear), with features X_t.

We can extend probabilistic and margin-based methods to multi-class settings:

- Multinomial logit (softmax probabilities)
- One-vs-rest or one-vs-one SVM

In this subsection we introduce two additional techniques that are naturally suited to multi-class classification: K-NN and Classification Trees. Both methods can also be applied in binary settings.

Outline

1. Introduction
2. Binary Classification
3. Multi Class Classification
   - k-Nearest Neighbors (k-NN)
   - CART - Classification and Regression Tree

What is the Probability of a Bear Market? (k-NN)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

What is the Probability of a Bear Market? (k-NN)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Our goal remains:

$$p_t = P(Y_t = 1 \mid x_t) = \mathbb{E}[Y_t \mid x_t]$$

What is the Probability of a Bear Market? (k-NN)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Our goal remains:

$$p_t = P(Y_t = 1 \mid x_t) = \mathbb{E}[Y_t \mid x_t]$$

Key idea (k-NN):
Instead of estimating a global model, we estimate this expectation locally:

$$\mathbb{E}[Y \mid X = x_t] \approx \text{average of } Y_i \text{ for observations with } x_i \approx x_t$$

What is the Probability of a Bear Market? (k-NN)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Our goal remains:

$$p_t = P(Y_t = 1 \mid x_t) = \mathbb{E}[Y_t \mid x_t]$$

Key idea (k-NN):
Instead of estimating a global model, we estimate this expectation locally:

$$\mathbb{E}[Y \mid X = x_t] \approx \text{average of } Y_i \text{ for observations with } x_i \approx x_t$$

- Training data: $\{(x_i, y_i)\}_{i=1}^N$

What is the Probability of a Bear Market? (k-NN)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Our goal remains:

$$p_t = P(Y_t = 1 \mid x_t) = \mathbb{E}[Y_t \mid x_t]$$

Key idea (k-NN):
Instead of estimating a global model, we estimate this expectation locally:

$$\mathbb{E}[Y \mid X = x_t] \approx \text{average of } Y_i \text{ for observations with } x_i \approx x_t$$

- Training data: $\{(x_i, y_i)\}_{i=1}^N$
- No parameters are estimated

What is the Probability of a Bear Market? (k-NN)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Our goal remains:

$$p_t = P(Y_t = 1 \mid x_t) = \mathbb{E}[Y_t \mid x_t]$$

Key idea (k-NN):
Instead of estimating a global model, we estimate this expectation locally:

$$\mathbb{E}[Y \mid X = x_t] \approx \text{average of } Y_i \text{ for observations with } x_i \approx x_t$$

- Training data: $\{(x_i, y_i)\}_{i=1}^N$
- No parameters are estimated
- The dataset itself serves as the model

What is the Probability of a Bear Market? (k-NN)

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Our goal remains:

$$p_t = P(Y_t = 1 \mid x_t) = \mathbb{E}[Y_t \mid x_t]$$

Key idea (k-NN):
Instead of estimating a global model, we estimate this expectation locally:

$$\mathbb{E}[Y \mid X = x_t] \approx \text{average of } Y_i \text{ for observations with } x_i \approx x_t$$

- Training data: $\{(x_i, y_i)\}_{i=1}^N$
- No parameters are estimated
- The dataset itself serves as the model

Interpretation: If nearby historical feature states were often Bear, then x_t is likely Bear.

k-NN Classification

We begin with a historical dataset

$$(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)$$

k-NN Classification

We begin with a historical dataset

$$(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)$$

To estimate the probability of Bear at x_t:

1. Compute the distance from x_t to all historical observations

$$d(x_t, x_i)$$

k-NN Classification

We begin with a historical dataset

$$(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)$$

To estimate the probability of Bear at x_t:

1. Compute the distance from x_t to all historical observations

$$d(x_t, x_i)$$

2. Identify the k closest observations

$$\mathcal{N}_k(x_t)$$

k-NN Classification

We begin with a historical dataset

$$(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)$$

To estimate the probability of Bear at x_t:

1. Compute the distance from x_t to all historical observations

$$d(x_t, x_i)$$

2. Identify the k closest observations

$$\mathcal{N}_k(x_t)$$

3. Estimate probability using their outcomes

$$\hat{p}(x_t) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x_t)} \mathbf{1}\{y_i = 1\}$$

k-NN Classification

We begin with a historical dataset

$$(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)$$

To estimate the probability of Bear at x_t:

1. Compute the distance from x_t to all historical observations

$$d(x_t, x_i)$$

2. Identify the k closest observations

$$\mathcal{N}_k(x_t)$$

3. Estimate probability using their outcomes

$$\hat{p}(x_t) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x_t)} \mathbf{1}\{y_i = 1\}$$

Classification rule:

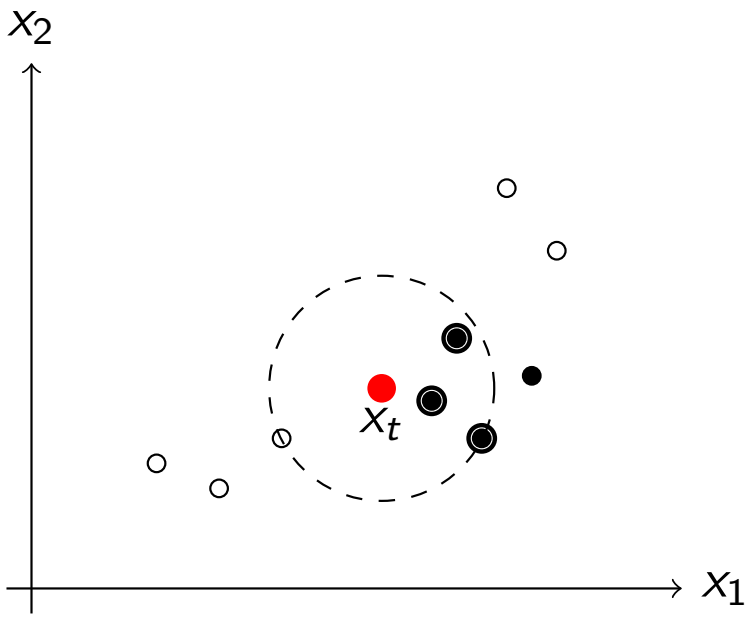
$$\hat{Y}_t = 1 \quad \text{if} \quad \hat{p}(x_t) \geq \tau$$

k-NN: geometric intuition

Each point represents a historical observation (x_i, y_i) in feature space.

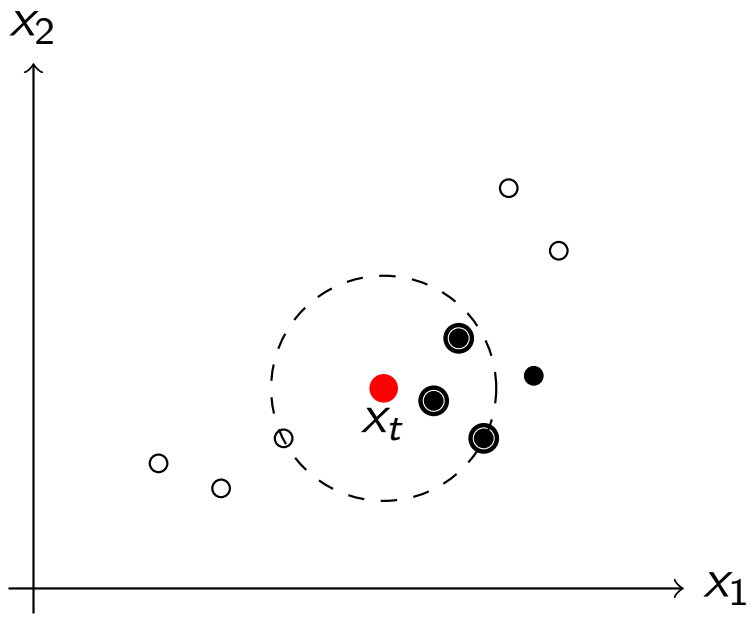
k-NN: geometric intuition

Each point represents a historical observation (x_i, y_i) in feature space.



k-NN: geometric intuition

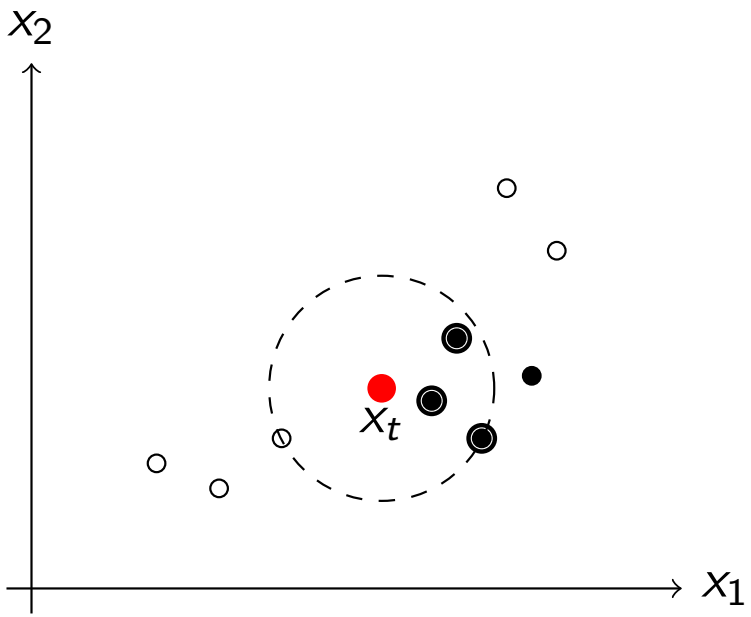
Each point represents a historical observation (x_i, y_i) in feature space.



- White circles: historical Not-Bear observations ($y_i = 0$).

k-NN: geometric intuition

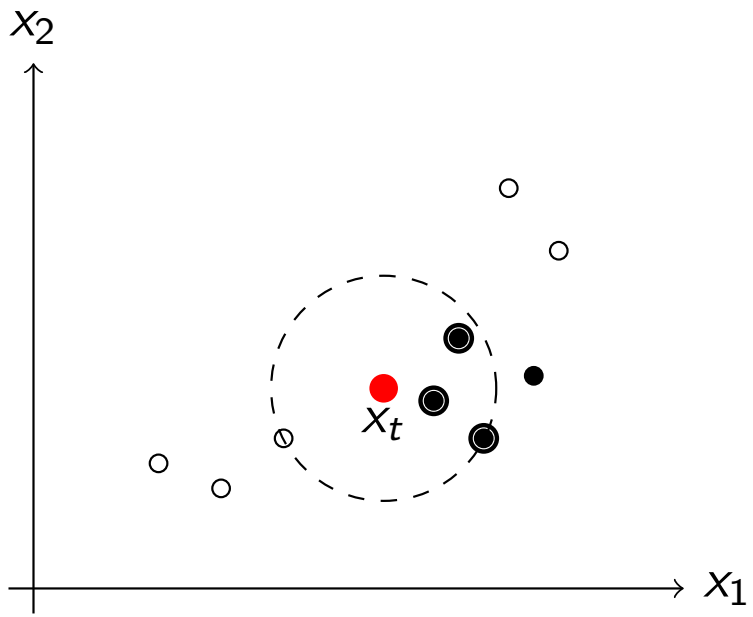
Each point represents a historical observation (x_i, y_i) in feature space.



- White circles: historical Not-Bear observations ($y_i = 0$).
- Black circles: historical Bear observations ($y_i = 1$).

k-NN: geometric intuition

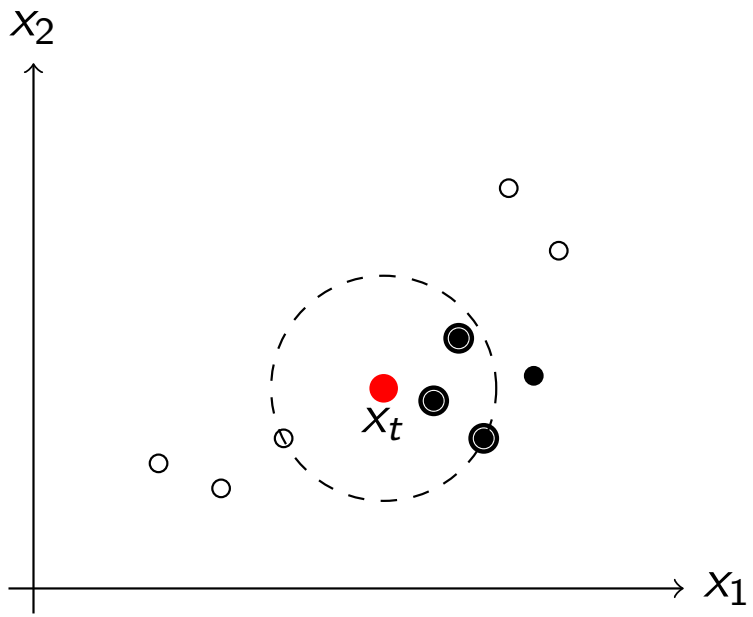
Each point represents a historical observation (x_i, y_i) in feature space.



- White circles: historical Not-Bear observations ($y_i = 0$).
- Black circles: historical Bear observations ($y_i = 1$).
- Red point x_t: the feature vector we want to classify using historical data.

k-NN: geometric intuition

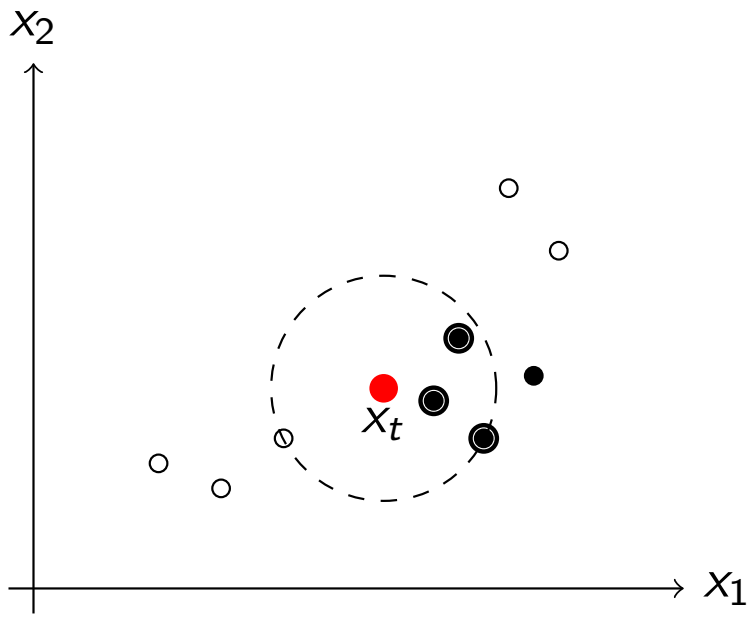
Each point represents a historical observation (x_i, y_i) in feature space.



- White circles: historical Not-Bear observations ($y_i = 0$).
- Black circles: historical Bear observations ($y_i = 1$).
- Red point x_t: the feature vector we want to classify using historical data.
- Dashed circle identifies the k nearest neighbors $\mathcal{N}_k(x_t)$, which expands until contains exactly k neighbors.

k-NN: geometric intuition

Each point represents a historical observation (x_i, y_i) in feature space.



- White circles: historical Not-Bear observations ($y_i = 0$).
- Black circles: historical Bear observations ($y_i = 1$).
- Red point x_t: the feature vector we want to classify using historical data.
- Dashed circle identifies the k nearest neighbors $\mathcal{N}_k(x_t)$, which expands until contains exactly k neighbors.
- The predicted class is determined by the majority class among those neighbors.

k-NN Hyperparameter Tuning

The key hyperparameter is

$$k = \text{number of neighbors}$$

k-NN Hyperparameter Tuning

The key hyperparameter is

$$k = \text{number of neighbors}$$

Small k

k-NN Hyperparameter Tuning

The key hyperparameter is

$$k = \text{number of neighbors}$$

Small k

- Uses only very similar observations
- Flexible decision boundary
- Low bias, high variance

k-NN Hyperparameter Tuning

The key hyperparameter is

$$k = \text{number of neighbors}$$

Small k

- Uses only very similar observations
- Flexible decision boundary
- Low bias, high variance

Large k

k-NN Hyperparameter Tuning

The key hyperparameter is

$$k = \text{number of neighbors}$$

Small k

- Uses only very similar observations
- Flexible decision boundary
- Low bias, high variance

Large k

- Uses many observations
- Smoother decision boundary
- Higher bias, lower variance

k-NN Hyperparameter Tuning

The key hyperparameter is

$$k = \text{number of neighbors}$$

Small k

- Uses only very similar observations
- Flexible decision boundary
- Low bias, high variance

Large k

- Uses many observations
- Smoother decision boundary
- Higher bias, lower variance

In practice, k is chosen using cross-validation.

K-means vs k-NN

K-means	**k-NN**
Unsupervised (uses only X)	Supervised (uses (X, Y))
Learns K cluster centroids	No parametric training (stores data)
Assigns points to nearest centroid	Predicts label via nearest neighbors' labels
Output: cluster ID	Output: class label / class probability

Outline

1. Introduction
2. Binary Classification
3. Multi Class Classification
   - k-Nearest Neighbors (k-NN)
   - CART - Classification and Regression Tree

What is the Probability of a Bear Market?

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

What is the Probability of a Bear Market?

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Our goal remains

$$p_t = P(Y_t = 1 \mid x_t)$$

What is the Probability of a Bear Market?

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Our goal remains

$$p_t = P(Y_t = 1 \mid x_t)$$

Instead of estimating a parametric model, classification trees divide the feature space into regions.

What is the Probability of a Bear Market?

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Our goal remains

$$p_t = P(Y_t = 1 \mid x_t)$$

Instead of estimating a parametric model, classification trees divide the feature space into regions.
Within each region, the probability of Bear is estimated using historical frequencies:

$$\hat{p}_t = \frac{\#\{\text{Bear observations in the region}\}}{\#\{\text{observations in the region}\}}$$

What is the Probability of a Bear Market?

Let

$$Y_t = \begin{cases} 1 & \text{bear market at time } t \\ 0 & \text{not-bear at time } t \end{cases} \qquad x_t \in \mathbb{R}^K$$

Our goal remains

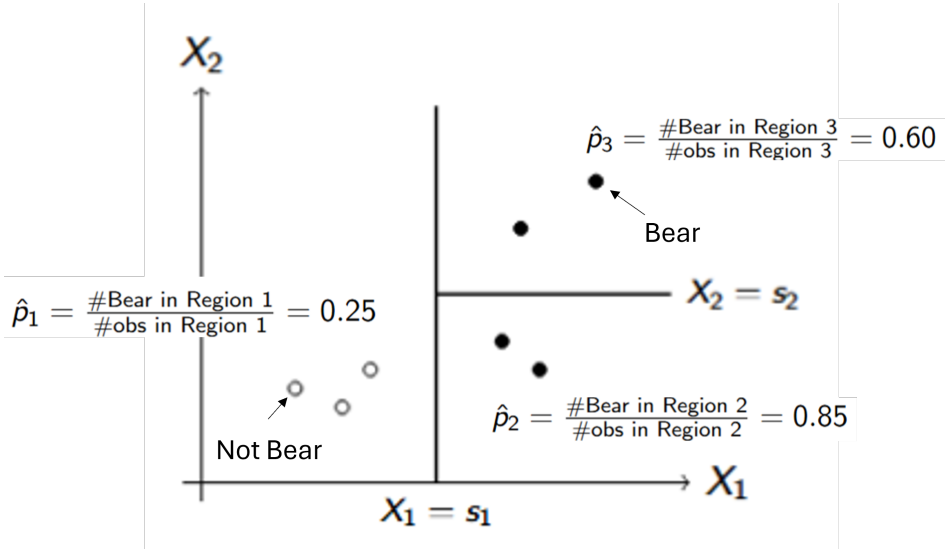
$$p_t = P(Y_t = 1 \mid x_t)$$

Instead of estimating a parametric model, classification trees divide the feature space into regions.
Within each region, the probability of Bear is estimated using historical frequencies:

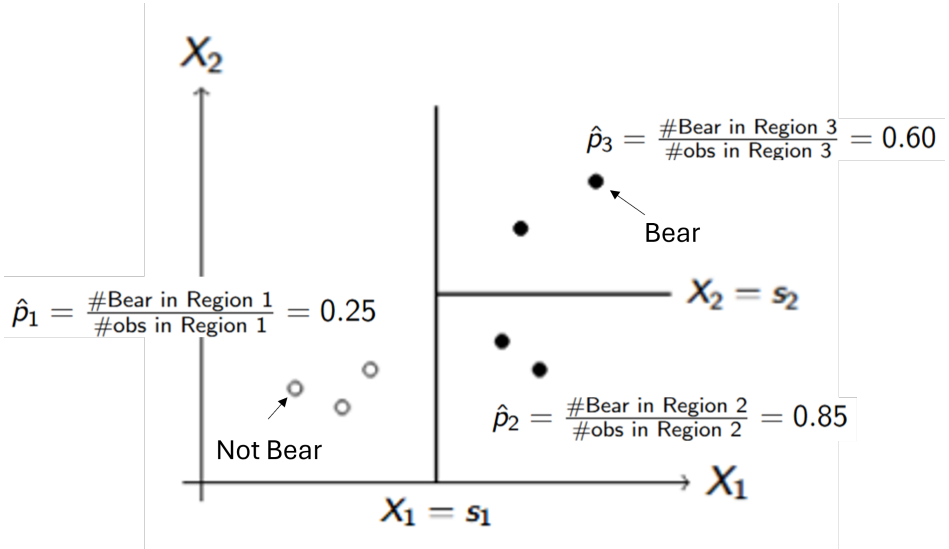
$$\hat{p}_t = \frac{\#\{\text{Bear observations in the region}\}}{\#\{\text{observations in the region}\}}$$

The tree learns a sequence of decision rules that determine which region x_t belongs to.

Splitting the Feature Space

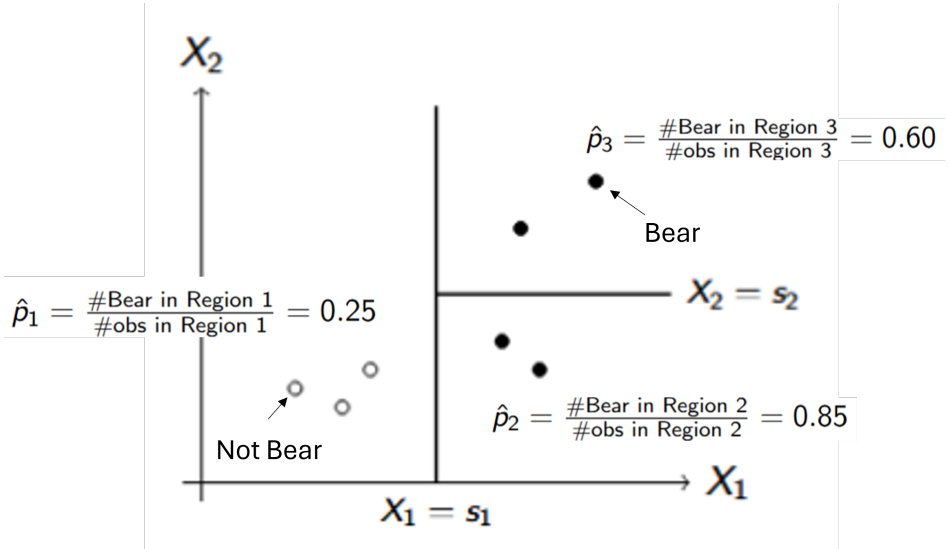


Splitting the Feature Space



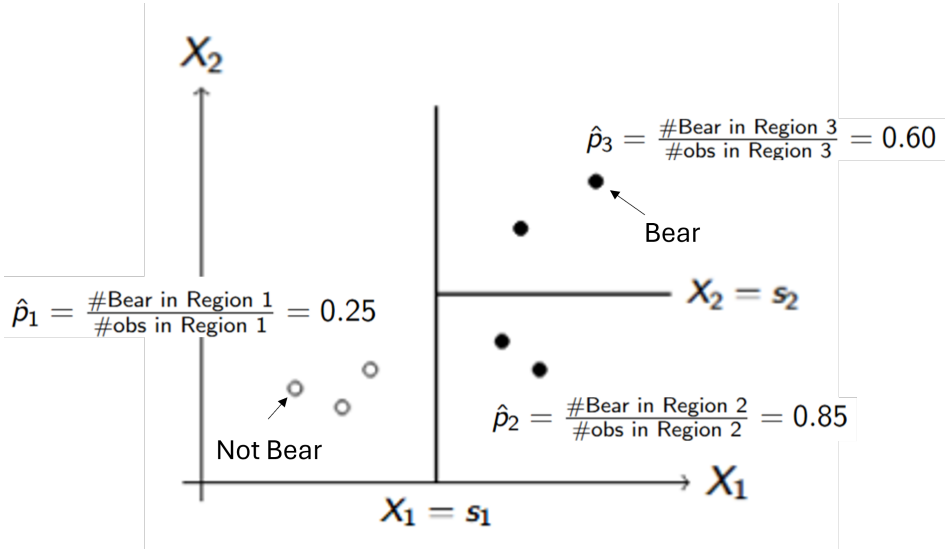
- CART splits the feature space using simple decision rules.

Splitting the Feature Space



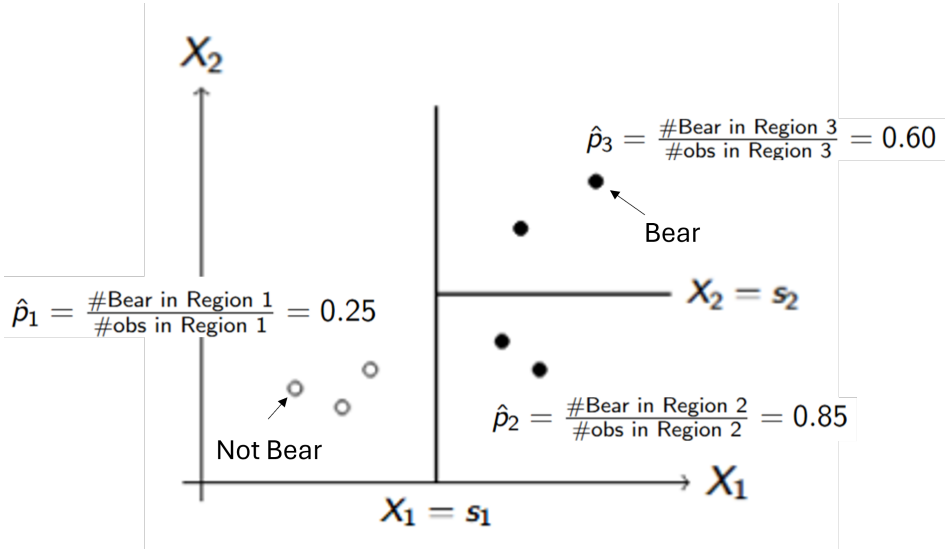
- CART splits the feature space using simple decision rules.
- Each split divides a region into two subregions based on a feature threshold.

Splitting the Feature Space



- CART splits the feature space using simple decision rules.
- Each split divides a region into two subregions based on a feature threshold.
- The algorithm recursively partitions the space to reduce classification impurity.

Splitting the Feature Space



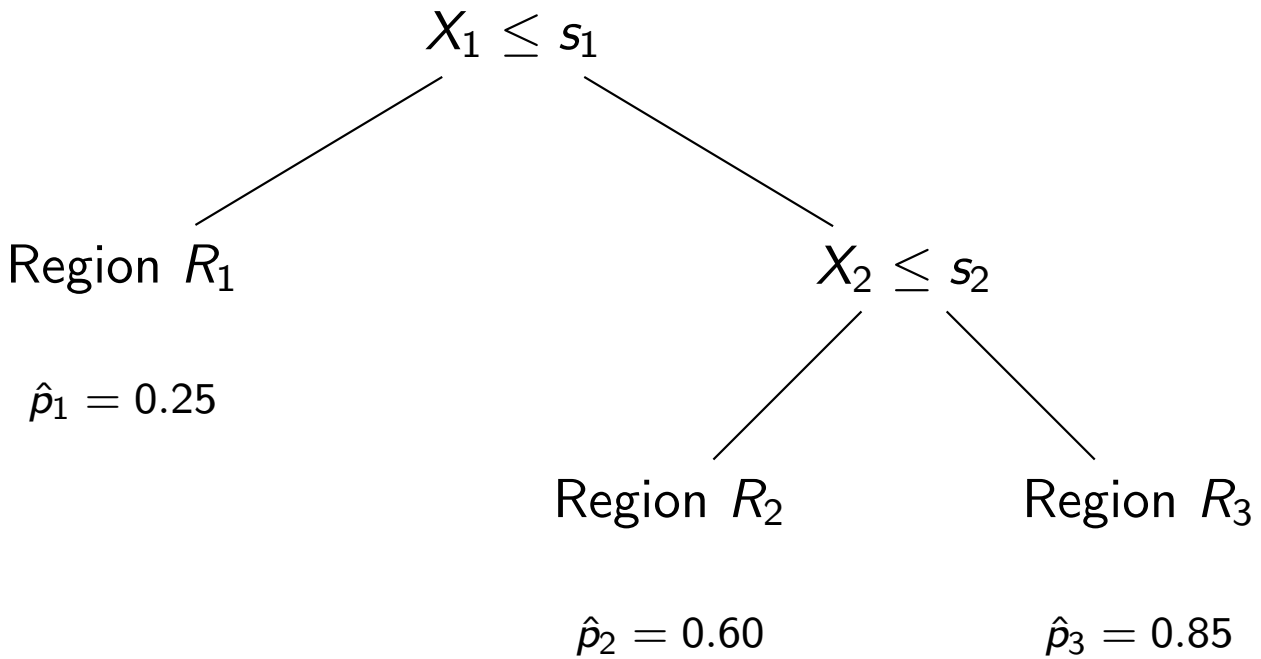
- CART splits the feature space using simple decision rules.
- Each split divides a region into two subregions based on a feature threshold.
- The algorithm recursively partitions the space to reduce classification impurity.
- Observations in the same region receive the same prediction.

Equivalent Tree Representation

The feature-space partition can be represented as a decision tree.

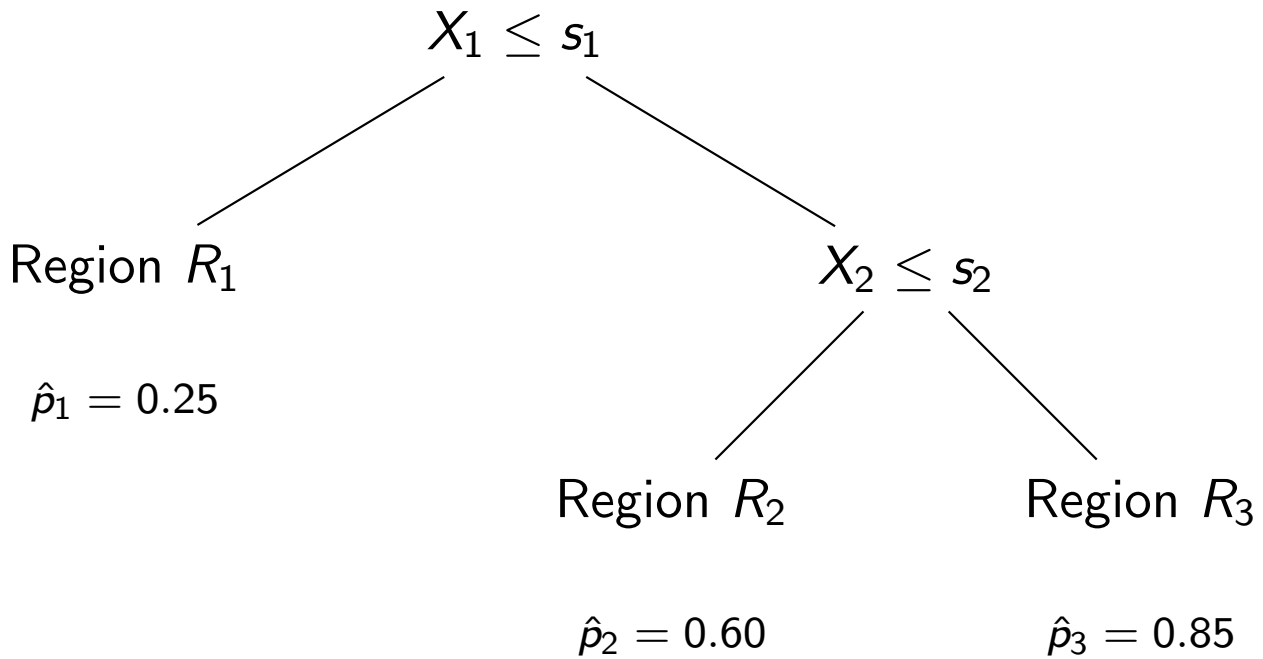
Equivalent Tree Representation

The feature-space partition can be represented as a decision tree.



Equivalent Tree Representation

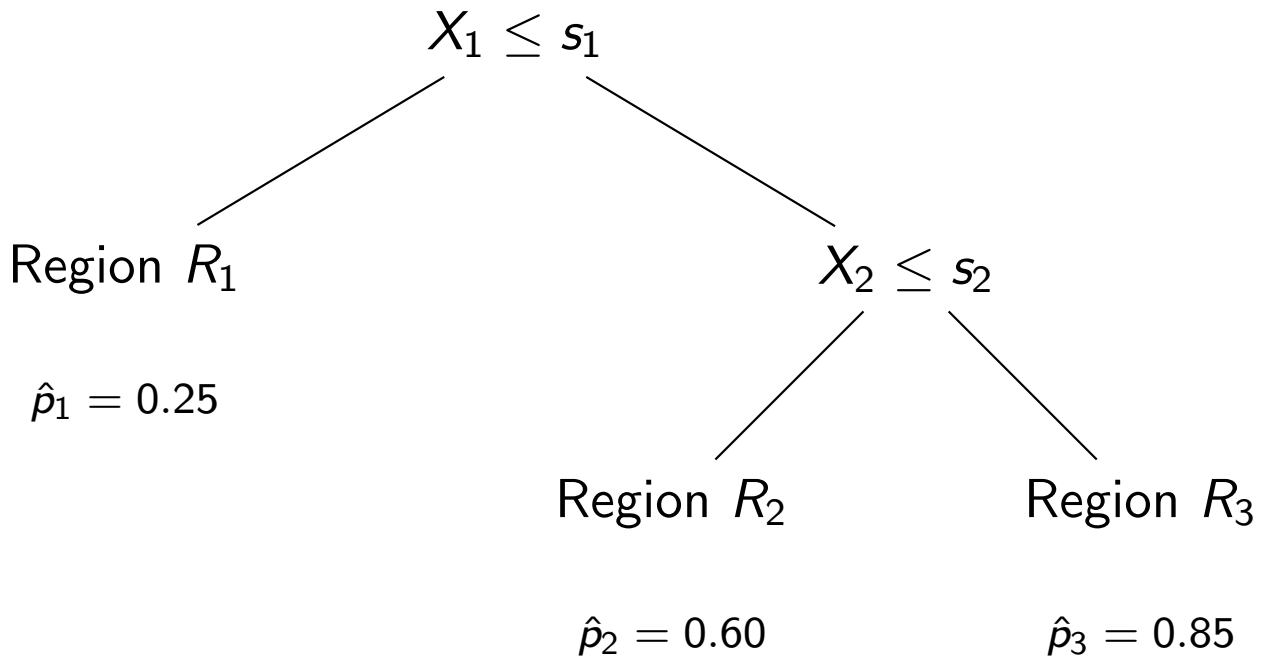
The feature-space partition can be represented as a decision tree.



- Each path down the tree defines a region in feature space.

Equivalent Tree Representation

The feature-space partition can be represented as a decision tree.



- Each path down the tree defines a region in feature space.
- Each terminal node produces a probability estimate of Bear.

How CART grows a classification tree

At each node:

1. For each feature j and candidate split point s, consider splitting the data:

$$\{X_j \leq s\} \text{ vs } \{X_j > s\}$$

How CART grows a classification tree

At each node:

1. For each feature j and candidate split point s, consider splitting the data:

$$\{X_j \leq s\} \text{ vs } \{X_j > s\}$$

2. Compute the reduction in impurity (e.g., Gini) from that split.

How CART grows a classification tree

At each node:

1. For each feature j and candidate split point s, consider splitting the data:

$$\{X_j \leq s\} \text{ vs } \{X_j > s\}$$

2. Compute the reduction in impurity (e.g., Gini) from that split.
3. Choose the best (j, s) and create two child nodes.

How CART grows a classification tree

At each node:

1. For each feature j and candidate split point s, consider splitting the data:

$$\{X_j \leq s\} \text{ vs } \{X_j > s\}$$

2. Compute the reduction in impurity (e.g., Gini) from that split.
3. Choose the best (j, s) and create two child nodes.
4. Recurse until stopping criteria are met (depth, min leaf size, etc.).

How CART grows a classification tree

At each node:

1. For each feature j and candidate split point s, consider splitting the data:

$$\{X_j \leq s\} \text{ vs } \{X_j > s\}$$

2. Compute the reduction in impurity (e.g., Gini) from that split.
3. Choose the best (j, s) and create two child nodes.
4. Recurse until stopping criteria are met (depth, min leaf size, etc.).

The tree is grown greedily (locally optimal splits), not globally optimal.

Impurity for classification: Gini

For a node with class proportions $\{p_c\}_{c=1}^{C}$:

$$\text{Gini} = 1 - \sum_{c=1}^{C} p_c^2$$

Impurity for classification: Gini

For a node with class proportions $\{p_c\}_{c=1}^{C}$:

$$\text{Gini} = 1 - \sum_{c=1}^{C} p_c^2$$

Binary case ($p = p_1$):

$$\text{Gini} = 2p(1-p)$$

Impurity for classification: Gini

For a node with class proportions $\{p_c\}_{c=1}^{C}$:

$$\text{Gini} = 1 - \sum_{c=1}^{C} p_c^2$$

Binary case ($p = p_1$):

$$\text{Gini} = 2p(1-p)$$

- Gini $= 0$ implies

Impurity for classification: Gini

For a node with class proportions $\{p_c\}_{c=1}^{C}$:

$$\text{Gini} = 1 - \sum_{c=1}^{C} p_c^2$$

Binary case ($p = p_1$):

$$\text{Gini} = 2p(1-p)$$

- Gini $= 0$ implies node contains only 1 class

Impurity for classification: Gini

For a node with class proportions $\{p_c\}_{c=1}^{C}$:

$$\text{Gini} = 1 - \sum_{c=1}^{C} p_c^2$$

Binary case ($p = p_1$):

$$\text{Gini} = 2p(1-p)$$

- Gini $= 0$ implies node contains only 1 class
- Gini $= 0.5$ implies

Impurity for classification: Gini

For a node with class proportions $\{p_c\}_{c=1}^{C}$:

$$\text{Gini} = 1 - \sum_{c=1}^{C} p_c^2$$

Binary case ($p = p_1$):

$$\text{Gini} = 2p(1-p)$$

- Gini $= 0$ implies node contains only 1 class
- Gini $= 0.5$ implies node is maximally mixed (50/50)

Impurity for classification: Gini

For a node with class proportions $\{p_c\}_{c=1}^{C}$:

$$\text{Gini} = 1 - \sum_{c=1}^{C} p_c^2$$

Binary case ($p = p_1$):

$$\text{Gini} = 2p(1-p)$$

- Gini $= 0$ implies node contains only 1 class
- Gini $= 0.5$ implies node is maximally mixed (50/50)
- Gini measures how mixed the classes are in a node

Impurity for classification: Gini

For a node with class proportions $\{p_c\}_{c=1}^{C}$:

$$\text{Gini} = 1 - \sum_{c=1}^{C} p_c^2$$

Binary case ($p = p_1$):

$$\text{Gini} = 2p(1-p)$$

- Gini $= 0$ implies node contains only 1 class
- Gini $= 0.5$ implies node is maximally mixed (50/50)
- Gini measures how mixed the classes are in a node

We prefer splits that are as pure as possible, so CART chooses splits that maximize impurity reduction:

$$\Delta = \text{Gini}(\text{parent}) - \left(\frac{n_L}{n}\text{Gini}(L) + \frac{n_R}{n}\text{Gini}(R)\right)$$

where Parent = node before the split; L and R are the resulting child nodes.

Impurity for classification: Gini

For a node with class proportions $\{p_c\}_{c=1}^{C}$:

$$\text{Gini} = 1 - \sum_{c=1}^{C} p_c^2$$

Binary case ($p = p_1$):

$$\text{Gini} = 2p(1-p)$$

- Gini $= 0$ implies node contains only 1 class
- Gini $= 0.5$ implies node is maximally mixed (50/50)
- Gini measures how mixed the classes are in a node

We prefer splits that are as pure as possible, so CART chooses splits that maximize impurity reduction:

$$\Delta = \text{Gini}(\text{parent}) - \left(\frac{n_L}{n}\text{Gini}(L) + \frac{n_R}{n}\text{Gini}(R)\right)$$

where Parent = node before the split; L and R are the resulting child nodes.

Example: Gini Reduction from a Split

Parent node:

	Bear	Not-Bear
Count	5	5

Example: Gini Reduction from a Split

Parent node:

	Bear	Not-Bear
Count	5	5

$$\text{Gini(parent)} = 2 * (0.5)(1 - 0.5) = 0.5$$

Example: Gini Reduction from a Split

Parent node:

	Bear	Not-Bear
Count	5	5

$$\text{Gini(parent)} = 2 * (0.5)(1 - 0.5) = 0.5$$

Candidate split produces:

	Bear	Not-Bear
Left ($n_L = 5$)	4	1
Right ($n_R = 5$)	1	4

Example: Gini Reduction from a Split

Parent node:

	Bear	Not-Bear
Count	5	5

$$\text{Gini(parent)} = 2 * (0.5)(1 - 0.5) = 0.5$$

Candidate split produces:

	Bear	Not-Bear
Left ($n_L = 5$)	4	1
Right ($n_R = 5$)	1	4

$$\text{Gini}(L) = 2 * (4/5)(1 - 4/5) = 0.32, \quad \text{Gini}(R) = 2 * (4/5)(1 - 4/5) = 0.32$$

Example: Gini Reduction from a Split

Parent node:

	Bear	Not-Bear
Count	5	5

$$\text{Gini(parent)} = 2 * (0.5)(1 - 0.5) = 0.5$$

Candidate split produces:

	Bear	Not-Bear
Left ($n_L = 5$)	4	1
Right ($n_R = 5$)	1	4

$$\text{Gini}(L) = 2 * (4/5)(1 - 4/5) = 0.32, \quad \text{Gini}(R) = 2 * (4/5)(1 - 4/5) = 0.32$$

$$\text{Weighted impurity} = (5/10)(0.32) + (5/10)(0.32) = 0.32$$

Example: Gini Reduction from a Split

Parent node:

	Bear	Not-Bear
Count	5	5

$$\text{Gini(parent)} = 2 * (0.5)(1 - 0.5) = 0.5$$

Candidate split produces:

	Bear	Not-Bear
Left ($n_L = 5$)	4	1
Right ($n_R = 5$)	1	4

$$\text{Gini}(L) = 2 * (4/5)(1 - 4/5) = 0.32, \quad \text{Gini}(R) = 2 * (4/5)(1 - 4/5) = 0.32$$

$$\text{Weighted impurity} = (5/10)(0.32) + (5/10)(0.32) = 0.32$$

$$\Delta = 0.5 - 0.32 = 0.18 \ \rightarrow \text{Good split}$$

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data $\Rightarrow$ overfitting

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data $\Rightarrow$ overfitting

Two strategies are commonly used to control tree complexity.

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data ⇒ overfitting

Two strategies are commonly used to control tree complexity.

1. **Pre-pruning (early stopping)**

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data ⇒ overfitting

Two strategies are commonly used to control tree complexity.

1. **Pre-pruning (early stopping)**
   - Stop the tree from growing too large

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data ⇒ overfitting

Two strategies are commonly used to control tree complexity.

1. **Pre-pruning (early stopping)**
   - Stop the tree from growing too large
   - Typical hyperparameters:
     - maximum depth
     - minimum samples per leaf
     - minimum impurity decrease

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data $\Rightarrow$ overfitting

Two strategies are commonly used to control tree complexity.

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data $\Rightarrow$ overfitting

Two strategies are commonly used to control tree complexity.

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data ⇒ overfitting

Two strategies are commonly used to control tree complexity.

2. **Post-pruning**

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data ⇒ overfitting

Two strategies are commonly used to control tree complexity.

2. **Post-pruning**
   - First grow a large tree.

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data ⇒ overfitting

Two strategies are commonly used to control tree complexity.

2. **Post-pruning**
   - First grow a large tree.
   - Then prune it back using a complexity penalty.

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data ⇒ overfitting

Two strategies are commonly used to control tree complexity.

2. **Post-pruning**
   - First grow a large tree.
   - Then prune it back using a complexity penalty.
   - Cost-complexity pruning chooses the tree T that minimizes

$$\min_{T} \ \widehat{R}(T) + \alpha |T|$$

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data ⇒ overfitting

Two strategies are commonly used to control tree complexity.

2. **Post-pruning**
   - First grow a large tree.
   - Then prune it back using a complexity penalty.
   - Cost-complexity pruning chooses the tree T that minimizes

$$\min_T \ \widehat{R}(T) + \alpha |T|$$

   - $\widehat{R}(T) =$ training misclassification rate of tree T

$$\widehat{R}(T) = \sum_{m=1}^{|T|} \frac{N_m}{N} \left(1 - \max_k \hat{p}_{mk}\right)$$

Overfitting, Pruning, and Hyperparameters

A deep tree can perfectly fit the training data ⇒ overfitting

Two strategies are commonly used to control tree complexity.

2. **Post-pruning**
   - First grow a large tree.
   - Then prune it back using a complexity penalty.
   - Cost-complexity pruning chooses the tree T that minimizes

$$\min_T \ \widehat{R}(T) + \alpha|T|$$

   - $\widehat{R}(T)$ = training misclassification rate of tree T

$$\widehat{R}(T) = \sum_{m=1}^{|T|} \frac{N_m}{N} \left(1 - \max_k \hat{p}_{mk}\right)$$

   - $|T|$ = number of leaves
   - α controls the penalty for tree complexity
   - Choose α using validation or cross-validation

1. Introduction

2. Binary Classification
   - Linear Probability Model (LPM)
   - Binary Probabilistic Classification (Logit/Probit)
   - Support Vector Machines (SVM)
   - Goodness of Fit

3. Multi Class Classification
   - k-Nearest Neighbors (k-NN)
   - CART - Classification and Regression Tree